

CRFD: Causal and Interpretable Fault Diagnosis Using Counterfactual Reasoning for Microservices on Multi-source Observability Data

XIANGBO TIAN, School of Computer Science, Wuhan University, China

SHI YING*, School of Computer Science, Wuhan University, China

TIANGANG LI, School of Computer Science, Wuhan University, China

CHUAN SHI, School of Computer Science, Beijing University of Posts and Telecommunications, China

DING XIAO, School of Computer Science, Beijing University of Posts and Telecommunications, China

Causal and interpretable fault diagnosis is essential for ensuring the stable operation and efficient maintenance of large-scale microservice systems. However, the existing fault diagnosis approaches confront two significant challenges: (1) existing approaches usually suffer from a lack of causality, which leads to their tendency to attribute faults to superficial phenomena rather than investigating the root causes behind superficial phenomena. (2) existing approaches typically suffer from a lack of interpretability, which makes them difficult to provide effective guidance for subsequent troubleshooting. In this study, we propose CRFD, a causal and interpretable fault diagnosis approach based on multi-source observability data for microservice systems, which uses counterfactual reasoning to diagnose faults and provide effective guidance for subsequent troubleshooting. CRFD transforms multi-source observability data into Structural Causal Models (SCMs). Then, it uses a Causal Variational Graph Autoencoder to learn SCMs, and detects anomalies based on SCMs. When an anomaly is detected, counterfactual reasoning is performed based on SCMs to localize root causes and tell operators how to deal with them to effectively mitigate faults. Experimental results on two datasets show that CRFD outperforms all baselines in terms of anomaly detection and root cause localization.

CCS Concepts: • **Software and its engineering** → **Software reliability**; **Software maintenance tools**; • **Computing methodologies** → **Causal reasoning and diagnostics**.

Additional Key Words and Phrases: Microservice Systems, Fault Diagnosis, Anomaly Detection, Root Cause Localization, Structural Causal Model, Counterfactual Reasoning, Variational Graph Autoencoder

1 Introduction

Microservice architecture is being increasingly adopted by more and more cloud-native enterprises due to its deployment flexibility, technical diversity, high scalability and fault tolerance [13, 31, 37]. However, the high dynamism and complexity of microservice systems bring greater challenges for fault diagnosis. The complex interaction between loosely-coupled services allows anomalies to propagate freely to other services [13], which further increases the difficulty of fault diagnosis.

*Shi Ying is the corresponding author.

Authors' Contact Information: Xiangbo Tian, tianxiangbo@whu.edu.cn, School of Computer Science, Wuhan University, Wuhan, Hubei, China; Shi Ying, yingshi@whu.edu.cn, School of Computer Science, Wuhan University, Wuhan, Hubei, China; Tiangang Li, tiangangli@whu.edu.cn, School of Computer Science, Wuhan University, Wuhan, Hubei, China; Chuan Shi, shichuan@bupt.edu.cn, School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, Beijing, China; Ding Xiao, dxiao@bupt.edu.cn, School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, Beijing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART

<https://doi.org/10.1145/3796706>

To address this challenge, many studies [12, 13, 17–19, 30, 32, 36, 37] have been made. However, these studies continue to confront two significant challenges: (1) existing approaches usually suffer from a lack of causality. They tend to attribute faults to superficial phenomena, but cannot investigate the root causes behind superficial phenomena. (2) existing approaches typically suffer from a lack of interpretability. They can only provide potential root causes, but cannot tell operators why they are culprits and how to deal with them to effectively mitigate faults.

To overcome the above limitations, we propose CRFD, a causal and interpretable fault diagnosis approach based on multi-source observability data for microservice systems, which uses counterfactual reasoning to diagnose faults and provide effective guidance for subsequent troubleshooting. The key ideas are 1) transforming multi-source observability data to SCMs, 2) learning SCMs, 3) calculating normal probabilities of microservice systems based on SCMs, and 4) localizing root causes by counterfactual reasoning based on SCMs. Therefore, CRFD can not only leverage the meaningful information from multi-source observability data, but also achieve effective feedback and coordination between anomaly detection and root cause localization. More importantly, it can provide interpretable root causes and tell operators why they are culprits and how to deal with them to effectively mitigate faults.

Specifically, CRFD consists of five components: (1) SCM Constructor extracts events from the multi-source observability data of microservice systems, and constructs SCMs based on these events. (2) Pattern Analyzer extracts the normal behavior patterns from the multi-source observability data collected in the fault-free phase of microservice systems. (3) Causal Variational Graph Autoencoder learns SCMs and calculates their exogenous variables' representations. (4) Gaussian Mixture Distribution-based Anomaly Detector learns the normal behavior patterns of SCMs and uses it to calculate the normal probabilities of microservice systems. (5) Counterfactual Reasoning-based Root Cause Locator uses counterfactual reasoning based on SCMs to localize root causes, and tell operators why they are culprits and how to deal with them to effectively mitigate faults.

We conducted extensive experiments on two datasets, and experimental results show that CRFD outperforms all baselines in terms of anomaly detection and root cause localization. In summary, our contributions are given as follows:

- We present a novel approach that transforms the multi-source observability data of microservice systems into SCMs, which can describe the status of microservice systems from a causal perspective and facilitate subsequent fault diagnosis.
- We propose CRFD, a causal and interpretable fault diagnosis approach based on multi-source observability data for microservice systems, which uses counterfactual reasoning to diagnose faults of microservice systems and provide effective guidance for subsequent troubleshooting.
- We conduct extensive experiments on two datasets, and experimental results show that CRFD outperforms all baselines in terms of anomaly detection and root cause localization.

The rest of this article is organized as follows. Section 2 introduces the background and motivation of this study. Section 3 describes the specific details of CRFD. Section 4 shows the experimental results and analysis of CRFD. Section 5 introduces the related works. Section 6 concludes our work and gives future work.

2 Background and Motivation

2.1 Background

Microservice systems are usually distributed systems, which allow software developers to independently design functional units as services deployed on different machines [31, 39, 40]. Although microservice architecture provides larger reliability and flexibility for software development, the complex interaction between loosely-coupled services allows anomalies to propagate freely to other services [13], which further increases the difficulty of fault diagnosis. To accurately and timely diagnose faults of microservice systems, the operators must carefully

check three-pillar data of the observability of microservice systems, i.e., metrics, traces and logs [37]. These three-pillar data and their roles in fault diagnosis are introduced as follows.

Metric. Metrics are used to describe the status of microservice systems within a time interval, which consists of system-level metrics and user-perceived metrics [7, 37, 41]. The former can reflect the performance issues caused by insufficient resources, while the latter is usually just an external manifestation of the former. For example, excessive CPU utilization, memory utilization and network latency can all lead to slow response time. Therefore, we only concentrate on system-level metrics in this article.

Trace. Traces are used to describe the call paths of user requests [21]. A trace contains many spans, and each span represents a call process of service instances. These spans record the caller service, callee service, response time and call status code [37, 41]. Therefore, traces can reflect the service call faults and network faults.

Log. Logs are used to record the meaningful events during a service instance's duration, which are produced by the print sentence in source code [4, 20, 39]. They contain the timestamp, verbosity level and raw message [10]. Among them, timestamp records their execution time of events, verbosity level records their severity of events, and raw message records the specific information of events. Therefore, logs can reflect the execution faults of service instances.

2.2 Motivation

This section introduces four motivations for this work. The examples come from the dataset collected from a widely-used benchmark microservice system, TrainTicket [13, 31, 37, 39, 40, 45].

Motivation 1: Enhancing causality can facilitate accurate fault diagnosis and effective fault mitigation. In microservice systems, good causality can provide fine-grained information about the faults of microservice systems, which can help to effectively diagnose and mitigate faults. However, as shown in Table 1, most existing studies [12, 13, 17–19, 30, 32, 36, 37] for fault diagnosis can only localize root causes at service level or metric level rather than investigating the causes behind superficial phenomena. To solve this challenge, we advocate for the fault diagnosis with higher causality.

Table 1. Comparison of state-of-the-art fault diagnosis approaches.

Approach	Observability Data			Tasks		Interpretability		
	Metrics	Traces	Logs	AD	RCA	What	Why	How
DeepLog [4]	×	×	✓	✓	×	✓	✓	×
LogAnomaly [20]	×	×	✓	✓	×	✓	✓	×
DeepTraLog [39]	×	✓	✓	✓	×	✓	×	×
iTCRL-LOF [31]	×	✓	✓	✓	×	✓	×	×
TraceAnomaly [17]	×	✓	×	✓	Service	✓	×	×
TBAC [19]	✓	×	×	×	Service	✓	×	×
MonitorRank [12]	✓	×	×	×	Service	✓	×	×
CloudRanger [32]	✓	×	×	×	Service	✓	×	×
MicroRank [36]	×	✓	×	×	Service	✓	×	×
CausalRCA [34]	✓	×	×	×	Metric	✓	×	×
Nezha [37]	✓	✓	✓	×	Event	✓	✓	×
Eadro [13]	✓	✓	✓	✓	Service	✓	×	×
CRFD	✓	✓	✓	✓	Event	✓	✓	✓

Motivation 2: Enhancing interpretability can facilitate clear and effective guidance of troubleshooting. In microservice systems, interpretability is essential for fault diagnosis. The interpretability of fault diagnosis

is divided into three levels: what-level, why-level and how-level. Among them, what-level interpretability means to know whether faults occurred, why-level interpretability means to know why faults occurred, and how-level interpretability means to know how to mitigate faults. However, as shown in Table 1, most existing approaches [12, 13, 17–19, 30, 32, 36, 37] for fault diagnosis are at what level, which limits the understanding and mitigation of faults. To solve this challenge, we advocate for the fault diagnosis with how-level interpretability.

Motivation 3: Effective coordination between anomaly detection and root cause localization can facilitate accurate fault diagnosis. Fault diagnosis for microservice systems contains two closely correlated phases: anomaly detection and root cause localization [13]. The former is the basis and trigger of the latter. However, most existing approaches lack effective feedback and coordination between anomaly detection and root cause localization, which is not conducive to accurate fault diagnosis. To solve this problem, we advocate for effective feedback and coordination between anomaly detection and root cause localization.

Motivation 4: Leveraging multi-source observability data can provide a comprehensive perspective for fault diagnosis. In microservice systems, different observability data contain different information, which play different roles in fault diagnosis. As shown in Table 2, different faults are reflected in different observability data, and no observability data alone can distinguish all faults. For example, metrics can reflect performance issues caused by insufficient resources, traces can reflect service call faults and network faults, and logs can reflect execution faults of service instances. To comprehensively diagnose faults, we advocate for an integration of multi-source observability data.

Table 2. Detailed information on faults in the TrainTicket dataset.

Metric	Trace	Log	Root Cause	Number
-	-	ERROR draw back money failed	Code Defect	630
-	response_time ↑	-	Network Jam	612
cpu_usage ↑	-	-	CPU Exhaustion	493

2.3 Preliminaries

In this section, we introduce the background knowledge about this study, containing the Structural Causal Model, Counterfactual Reasoning and Variational Graph Autoencoder.

Structural Causal Model. Structural Causal Model (SCM) [30] is a mathematical framework for describing and analyzing causal relationships. SCMs use a Directed Acyclic Graph (DAG) to model causal relationships between variables. It can not only help us understand direct causal relationships between variables, but also support causal effect prediction. In microservice systems, various call relationships and impact relationships are essentially causal relationships. Thus, SCMs are suitable to model the multi-source observability data of microservice systems.

Counterfactual Reasoning. Counterfactual reasoning [9, 28, 30] is a thinking way about ‘what-if’, where it needs to answer how outcomes would change if past events or actions had varied. It can be computed by the Abduction-Action-Prediction process [23]. Among them, ‘Abduction’ is used to infer the potential causes of the observed data, ‘Action’ is used to change the past world, and ‘Prediction’ is used to predict counterfactual outcomes based on the possible causes and changed world [28]. In fault diagnosis of microservice systems, root cause localization is essentially a counterfactual attribution problem. Therefore, counterfactual reasoning is suitable for root cause localization of microservice systems.

Variational Graph Autoencoder. Variational Graph Autoencoder (VGAE) is an unsupervised learning model for graph-structured data. By combining a Graph Neural Network with a Variational Autoencoder, it can learn

embedding representations of graph-structured data in an unsupervised manner. In microservice systems, there are complex causal relationships in multi-source observability data, and the cost of data annotation is high. Therefore, VGAE is suitable to learn the multi-source observability data of microservice systems.

2.4 Problem Statement

Our objective is to design a causal and interpretable approach for fault diagnosis of microservice systems. It consists of two tasks: anomaly detection and root cause localization. For the first task, our goal is to timely detect anomalies and trigger the second task when an anomaly is detected. For the second task, our goal is to provide a ranked list of possible root causes and effective guidance for subsequent troubleshooting.

3 Approach

In this study, we propose CRFD, a causal and interpretable fault diagnosis approach based on multi-source observability data for microservice systems, which uses counterfactual reasoning to diagnose faults and provide effective guidance for subsequent troubleshooting. Fig. 1 shows the overall structure of CRFD. As shown in Fig. 1, CRFD consists of **Multi-source Observability Data-based Pattern Learner (MDPL)** and **Counterfactual Reasoning-based Root Cause Locator (CRCL)**. The former learns the normal behavior patterns of the multi-source observability data collected in the fault-free phase, and timely detects anomalies in the fault-suffering phase. When an anomaly is detected, the latter is triggered to localize root causes.

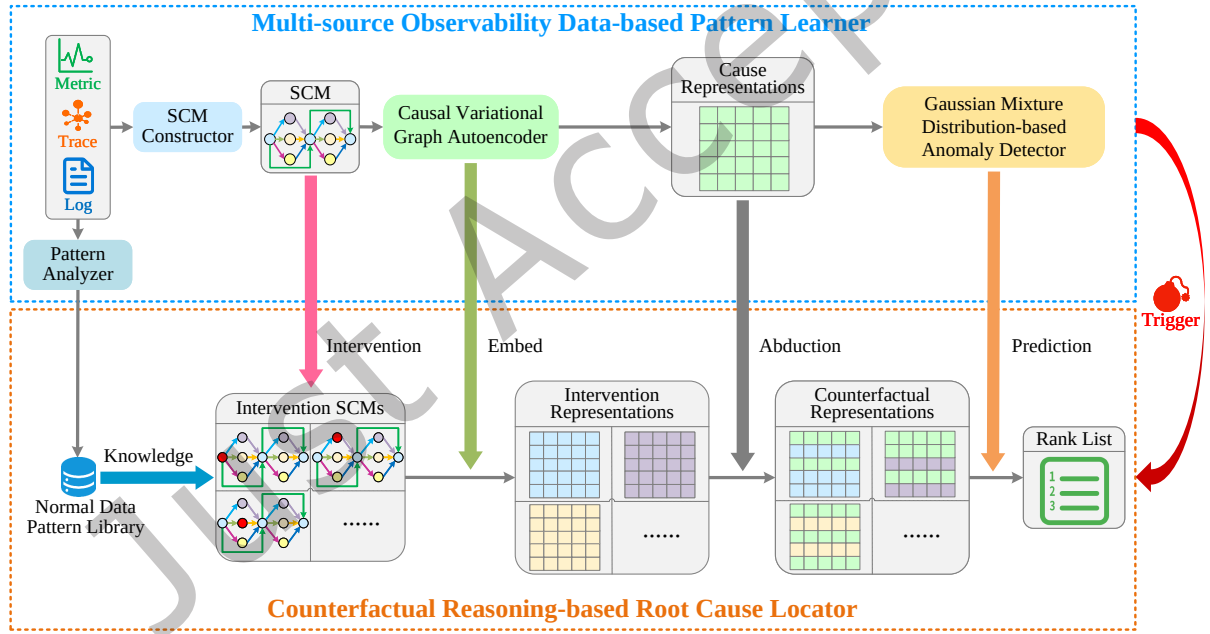


Fig. 1. Overview of CRFD.

In Multi-source Observability Data-based Pattern Learner, (1) **SCM Constructor** extracts events from the multi-source observability data of microservice systems, and constructs SCMs based on these events. (2) **Pattern Analyzer** learns the normal behavior patterns of the multi-source observability data collected in the fault-free phase of microservice systems. (3) **Causal Variational Graph Autoencoder** learns SCMs and calculates their

exogenous variables' representations. (4) **Gaussian Mixture Distribution-based Anomaly Detector** learns the normal behavior patterns of SCMs and uses it to calculate the normal probabilities of microservice systems.

In Counterfactual Reasoning-based Root Cause Locator, an Abduction-Action-Prediction process customized to microservice systems is designed for root cause localization, which can tell operators why root causes are culprits and how to deal with them to effectively mitigate faults.

3.1 Multi-source Observability Data-based Pattern Learner

3.1.1 SCM Constructor. In microservice systems, numerous observability data is generated every day, which contains a large number of events and complex causal relationships. They describe the status of microservice systems from different perspectives. Aggregating multi-source observability data based on events can provide a comprehensive and fine-grained perspective for fault diagnosis and troubleshooting. Therefore, SCM Constructor is designed to model multi-source observability data as SCMs. In distributed tracing, a span usually represents a complete invocation process of a service instance [21]. A span provides a coarse-grained description of the overall execution operation while implicitly covering multiple execution actions and interactions. However, this span-level information lacks fine-grained details on internal operations and resource status. To achieve the fine-grained modeling of microservice systems, we define events as basic units derived from traces, logs and metrics. Each event represents a fine-grained behavior of the corresponding span. In this study, the term 'event' has two semantic categories. The first is Status Events derived from metric data, which describe the runtime status of resources. The second is Flow Events derived from log data and trace data, which represent the execution and invocation behaviors of service instances. These events together describe the operational status and complex interactions of microservice systems, which serve as the basis for SCM construction. Formally, the definitions are given as follows:

- **Local Metric Event:** A Status Event $e = (s, t, r, m_s^{r,t})$ records the utilization status $m_s^{r,t}$ of local resource r allocated to a service instance s at time t .
- **Transmission Metric Event:** A Status Event $e = (s, q, t, r, m_{s,q}^{r,t})$ records the utilization status $m_{s,q}^{r,t}$ of transmission resource r of a pair of service instances (s, q) at time t .
- **Execution Event:** A Flow Event $e = (s, t, a)$ records an execution action a of a service instance s at time t .
- **Invocation Event:** A Flow Event $e = (s, t, a)$ records an invocation action a of a service instance s at time t , which can occur on either the client or server side, corresponding to the request and response of a service invocation.

Based on the above definitions, SCM Constructor is divided into three steps: **Event Extraction**, **Event Representation** and **SCM Construction**.

Event Extraction is designed to extract events from multi-source observability data. For metrics, it transforms each system-level metric into a Local Metric Event or Transmission Metric Event based on its resource type. For traces, each span is decomposed into two Invocation Events: a client-side invocation event and a server-side invocation event. The former represents the outgoing call, and the latter denotes the corresponding receiving action of the callee service instance. These paired invocation events are connected to construct cross-span causal relationships, and their actions are defined by the static templates of the corresponding api address. For logs, each log record is treated as an Execution Event, and its static template is used as its action. Based on the above processing, the four types of events form four event sets $E^{(L)}$, $E^{(T)}$, $E^{(E)}$ and $E^{(I)}$, which correspond to the sets of Local Metric Events, Transmission Metric Events, Execution Events and Invocation Events, respectively. Note that all templates are extracted by the state-of-the-art log parsing approach Drain [10].

Event Representation is designed to transform events into vectorized semantic representations. For Execution Events and Invocation Events, we use the pre-trained GloVe embeddings trained on the Common Crawl corpus

with 300-dimensional vectors, together with TF-IDF [27], to calculate their vector representations. For Metric Events, their vector representations are directly generated, where each dimension represents a metric.

SCM Construction is designed to construct an SCM $\mathcal{G} = \{U, V, A, X\}$ based on the above-mentioned events, where U represents the exogenous variable set, V is the endogenous variable set containing all above-mentioned events, A denotes the causal relationship set and X represents the feature map. The specific steps are given as follows.

- **Connection of Execution Events.** For Execution Events within each span, they are ordered by their timestamps to reflect the local execution order. Then, ‘Event-Event Causal’ relationships are added from an Execution Event to the next one.
- **Insertion of Invocation Events.** For each span, it contains paired Invocation Events on the client and server sides, which describe an outgoing call and its corresponding receiving action. Note that timestamp ordering is used only within each span to reflect local execution order, and cross-span causal relationships are established via paired Invocation Events to ensure correct modeling in asynchronous systems. These events are inserted into the sorted Execution Event sequence of their spans according to timestamps. Then, ‘Event-Event Causal’ relationships are added between Invocation Events and their neighboring events within a span, as well as between paired Invocation Events across caller-callee spans to model cross-span causal relationships.
- **Insertion of Metric Events.** For any two connected events, Local Metric Events are found based on the service instances and timestamp of the predecessor event, while Transmission Metric Events are found based on the service instances and timestamps of two related events. Then, ‘Event-Resource Causal’ relationships are added from the predecessor event to the associated Metric Events, and ‘Resource-Event Causal’ relationships are added from the associated Metric Events to the successor event. Note that ‘Resource’ in two causal relationships consists of multiple types, such as CPU, Memory and Network.
- **Insertion of Exogenous Variables.** For each event, an exogenous variable is generated, and ‘Exogenous Causal’ relationships are added between events and their corresponding exogenous variables.

Based on the above principles, we formalize the SCM Construction process in Algorithm 1, which illustrates the precise rules for node and edge creation, metric association, and time constraints. Note that an SCM is constructed for each trace.

Fig. 2 shows two samples of SCMs. Green rounded rectangles represent Execution Events, while blue rounded rectangles represent Invocation Events. Ellipses describe Metric Events. Gray circles denote exogenous variables. Arrows with different colors represent different causal relationships. Note that the dotted rounded rectangles are not nodes in SCMs but are just for resource description. These graphs can well describe the complex causal relationships in multi-source observability data, which facilitate accurate fault diagnosis and effective fault mitigation.

3.1.2 Pattern Analyzer. In microservice systems, multi-source observability data follows a latent behavior pattern, which leads their SCMs to also exhibit inherent rules. Therefore, the counterfactual assumptions must be reasonable and exist in the fault-free phase of real microservice systems. Therefore, Pattern Analyzer is designed to extract these inherent rules.

For Execution Events and Invocation Events in SCMs, Pattern Analyzer extracts each pair of directly connected events as an event pattern. The specific generation rule is given as follows.

$$\mathcal{P}_E = \{(e_i, e_j) \mid (e_i, e_j) \in A^V \wedge e_i, e_j \in E^{(I)} \cup E^{(E)}\} \quad (1)$$

where $\mathcal{P}_E$ is the event pattern library; e_i and e_j represent Execution Events or Invocation Events; $E^{(I)}$ and $E^{(E)}$ denote the sets of Invocation Events and Execution Events.

Algorithm 1: SCM Construction

Input: Trace τ ; Event set $\{E^{(L)}, E^{(T)}, E^{(E)}, E^{(I)}\}$; Event features $\{F^{(L)}, F^{(T)}, F^{(E)}, F^{(I)}\}$; Event service function $\phi(\cdot)$; Event timestamp function $T(\cdot)$; Event resource type function $\psi(\cdot)$

Output: SCM $\mathcal{G}$ of trace τ

```

1   $U, V, A, X \leftarrow \emptyset;$ 
   /* Connection of Execution Events */
2  for  $s \in \phi(\tau)$  do
3       $L_s \leftarrow$  sort  $E_{\tau,s}^{(E)}$  according to timestamps;
4      for  $i = 2$  to  $|L_s|$  do
5          if  $L_{s,i-1} \notin V$  then
6               $V \leftarrow V \cup \{L_{s,i-1}\};$  // Creation of log event nodes
7               $X^V[L_{s,i-1}] \leftarrow F^{(E)}[L_{s,i-1}];$  // Initiation of log node attributes
8          if  $L_{s,i} \notin V$  then
9               $V \leftarrow V \cup \{L_{s,i}\};$  // Creation of log event nodes
10              $X^V[L_{s,i}] \leftarrow F^{(E)}[L_{s,i}];$  // Initiation of log node attributes
11              $A^V[L_{s,i-1}, L_{s,i}] \leftarrow 0;$  // Add event-event causal relationships
   /* Insertion of Invocation Events */
12  for  $e \in E_{\tau}^{(I)}$  do
13      if  $e \notin V$  then
14           $V \leftarrow V \cup \{e\};$  // Creation of trace event nodes
15           $X^V[e] \leftarrow F^{(I)}[e];$  // Initiation of trace node attributes
16       $s \leftarrow \phi(e);$ 
17       $k \leftarrow \max\{j \mid T(L_{s,j}) < T(e)\};$ 
18       $A^V[L_{s,k}, e], A^V[e, L_{s,k+1}] \leftarrow 0;$  // Add event-event causal relationships
19       $C \leftarrow$  child service instance set of  $s$ ;
20      for  $c \in C$  do
21           $i \leftarrow \max\{j \mid T(L_{c,j}) < T(e)\};$ 
22           $A^V[L_{c,i}, e], A^V[e, L_{c,i+1}], A^V[L_{s,k+1}, L_{c,i+1}] \leftarrow 0;$  // Add event-event causal relationships
   /* Insertion of Metric Events */
23  for  $(e_p, e_c) \in A$  do
24       $M^{(L)} \leftarrow$  Local Metric Events from  $E_{\phi(e_p)}^{(L)}$  according to  $T(e_p)$ ;
25       $M^{(T)} \leftarrow$  Transmission Metric Events from  $E_{\phi(e_p), \phi(e_c)}^{(T)}$  according to  $T(e_p)$ ;
26      for  $m \in M^{(L)} \cup M^{(T)}$  do
27          if  $m \notin V$  then
28               $V \leftarrow V \cup \{m\};$  // Creation of metric event nodes
29              if  $m \in M^{(L)}$  then
30                   $X^V[m] \leftarrow F^{(L)}[m];$  // Initiation of metric node attributes
31              else
32                   $X^V[m] \leftarrow F^{(T)}[m];$  // Initiation of metric node attributes
33               $A^V[e_p, m] \leftarrow 2 \times \psi(m) + 2;$  // Add event-resource causal relationships
34               $A^V[m, e_c] \leftarrow 2 \times \psi(m) + 3;$  // Add resource-event causal relationships
   /* Insertion of Exogenous Variables */
35  for  $e \in V$  do
36       $U \leftarrow$  add an exogenous variable  $u$  into  $U$ ; // Creation of exogenous variables
37       $X^U[u] \leftarrow \emptyset;$  // Initiation of exogenous variable attributes
38       $A^U[u, e] \leftarrow 1;$  // Add exogenous causal relationships
39   $\mathcal{G} \leftarrow \{U, V, A, X\};$ 
40  return  $\mathcal{G};$ 

```

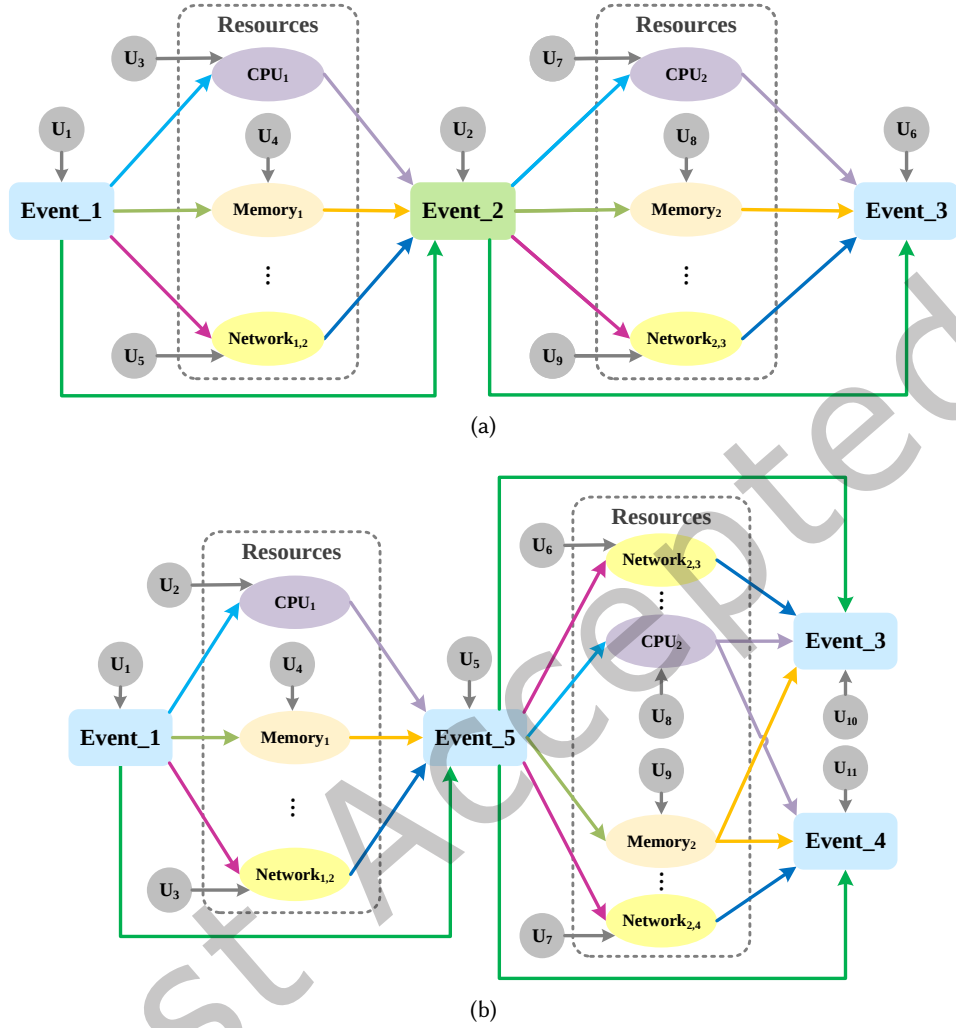


Fig. 2. Two examples of SCMs.

For Local Metric Events in SCMs, Pattern Analyzer calculates the mode value, minimum value and maximum value of each local resource for each service instance. The specific generation rule is given as follows.

$$\mathcal{P}_L(s, r, \Gamma) = \Gamma(m_s^r) \quad (2)$$

where $\mathcal{P}_L$ is the local metric pattern library; Γ denotes the *mode*, *min* or *max* operation; s represents a service instance; r denotes a local resource; m_s^r is the local resource r 's metric sequence of service instance s .

For Transmission Metric Events in SCMs, it calculates the mode value, minimum value and maximum value of each transmission resource for a pair of service instances. The specific generation rule is given as follows.

$$\mathcal{P}_T(s, q, r, \Gamma) = \Gamma(m_{s,q}^r) \quad (3)$$

where $\mathcal{P}_T$ is the transmission metric pattern library; Γ denotes the *mode*, *min* or *max* operation; (s, q) represents a pair of service instances; r denotes a transmission resource; $m_{s,q}^r$ is the resource r 's metric sequence of a pair of service instances (s, q) .

Finally, the above three patterns are stored in Normal Data Pattern Library (NDPL).

3.1.3 Causal Variational Graph Autoencoder. In an SCM $\mathcal{G}$, each endogenous variable $V_i^{\mathcal{G}}$ can be expressed as a function $\mathcal{F}_i$ of its exogenous variable $U_i^{\mathcal{G}}$ and causal parents $Pa(V_i^{\mathcal{G}})$, which is described by the following equation.

$$V_i^{\mathcal{G}} = \mathcal{F}_i(U_i^{\mathcal{G}}, Pa(V_i^{\mathcal{G}})) \quad (4)$$

Based on Equation 4, the exogenous variable $U_i^{\mathcal{G}}$ can be calculated by the following equation.

$$U_i^{\mathcal{G}} = \mathcal{F}_i^{-1}(V_i^{\mathcal{G}}, Pa(V_i^{\mathcal{G}})) \quad (5)$$

Therefore, exogenous variables can be calculated by their corresponding endogenous variables and the causal parents of these endogenous variables.

Inspired by VGAE, Causal Variational Graph Autoencoder (CVGA) is proposed to learn SCMs, which consists of four components: SCM Encoder, SCM Sampling, SCM Decoder and SCM Loss. Because nodes are updated synchronously in Graph Neural Networks while variables in SCMs are calculated sequentially according to their causal dependencies, stacking multiple convolutional layers in a CVGA can distort the causal relationships captured by SCMs. To ensure the correctness and interpretability of causal inference, SCM Encoder and SCM Decoder use only a single convolutional layer to simulate causal computation. This design preserves causal consistency between representation learning and causal computation, but sacrifices the ability to capture multi-hop fault propagation.

SCM Encoder aims to approximate the inverse causal function $\mathcal{F}_i^{-1}$, thereby learning the distribution of exogenous variables. In an SCM, endogenous variables and causal relationships have many types and lie in different feature spaces. Therefore, they need to be transformed into the same latent space before causal neighbor aggregation. For the endogenous variables in SCMs, a variable-type-specific linear transformation is designed to project the vector representations of endogenous variables into a latent space. The specific projection process is described by the following equation.

$$\mathbf{h}_i^{\mathcal{G}} = \mathbf{W}_{\varphi(V_i^{\mathcal{G}})} \mathbf{X}_i^{(\mathcal{G},V)} + \mathbf{b}_{\varphi(V_i^{\mathcal{G}})} \quad (6)$$

where $\mathbf{h}_i^{\mathcal{G}}$ is the projected vector representation of endogenous variable $V_i^{\mathcal{G}}$; $\mathbf{X}_i^{(\mathcal{G},V)}$ represents the vector representation of endogenous variable $V_i^{\mathcal{G}}$; $\mathbf{W}_{\varphi(V_i^{\mathcal{G}})}$ and $\mathbf{b}_{\varphi(V_i^{\mathcal{G}})}$ represent the learnable weight matrix and bias matrix of endogenous variable type $\varphi(V_i^{\mathcal{G}})$, respectively. Similar to endogenous variables, a relationship-type-specific linear transformation is designed to project different causal relationships into a latent space. The specific projection process is given by the following equation.

$$\mathbf{A}_{i,j}^{(\mathcal{G},V)'} = \text{LeakyReLU}(\mathbf{W}_E \mathbf{E}[\mathbf{A}_{i,j}^{(\mathcal{G},V)}] + \mathbf{b}_E) \quad (7)$$

where $\mathbf{A}_{i,j}^{(\mathcal{G},V)'}$ is the projected vector representation of the causal relationship from endogenous variable $V_j^{\mathcal{G}}$ to endogenous variable $V_i^{\mathcal{G}}$; $\mathbf{W}_E$ and $\mathbf{b}_E$ represent the learnable weight matrix and bias matrix of causal relationships, respectively; $\mathbf{E}$ denotes the learnable feature dictionary of causal relationships; $\mathbf{A}_{i,j}^{(\mathcal{G},V)}$ is the type of the causal relationship from endogenous variable $V_j^{\mathcal{G}}$ to endogenous variable $V_i^{\mathcal{G}}$. Then, the update message from each neighbor endogenous variable is calculated based on their semantic information and causal information. The

specific calculation process is given by the following equation.

$$\mathbf{M}_{i,j}^{\mathcal{G}} = \mathbf{W}_M[\mathbf{h}_j^{\mathcal{G}} \parallel \mathbf{A}_{i,j}^{(\mathcal{G},V)}] + \mathbf{b}_M \quad (8)$$

where $\mathbf{M}_{i,j}^{\mathcal{G}}$ is the update message from endogenous variable $V_j^{\mathcal{G}}$ to endogenous variable $V_i^{\mathcal{G}}$; $\mathbf{W}_M$ and $\mathbf{b}_M$ represent the learnable weight matrix and bias matrix, respectively. Afterwards, the causal weights of causal relationships are calculated to reflect the importance of different causal relationships. The specific calculation process is shown by the following equations.

$$a_{i,j}^{\mathcal{G}} = \text{LeakyReLU}\left(\mathbf{W}_A[\mathbf{h}_i^{\mathcal{G}} \parallel \mathbf{h}_j^{\mathcal{G}} \parallel \mathbf{A}_{i,j}^{(\mathcal{G},V)}]\right) \quad (9)$$

$$\alpha_{i,j}^{\mathcal{G}} = \frac{\exp(a_{i,j}^{\mathcal{G}})}{\sum_{k \in Pa(V_i^{\mathcal{G}})} \exp(a_{i,k}^{\mathcal{G}})} \quad (10)$$

where $a_{i,j}^{\mathcal{G}}$ is the causal weight of the causal relationship from endogenous variable $V_j^{\mathcal{G}}$ to endogenous variable $V_i^{\mathcal{G}}$; $\mathbf{W}_A$ represents the learnable weight matrix; $\alpha_{i,j}^{\mathcal{G}}$ denotes the normalized causal weight from endogenous variable $V_j^{\mathcal{G}}$ to endogenous variable $V_i^{\mathcal{G}}$. Finally, the vector representation $\mathbf{X}_i^{(\mathcal{G},U)}$ of exogenous variable $U_i^{\mathcal{G}}$ is calculated. The specific calculation process is given by the following equation.

$$\mathbf{X}_i^{(\mathcal{G},U)} = \mathbf{h}_i^{\mathcal{G}} + \sum_{j \in Pa(V_i^{\mathcal{G}})} \alpha_{i,j}^{\mathcal{G}} \times \mathbf{M}_{i,j}^{\mathcal{G}} \quad (11)$$

SCM Sampling aims to obtain samples from the distribution of exogenous variables. The specific sampling rules are shown by the following equations.

$$\mathbf{X}_i^{(\mathcal{G},U)} = [\mu_i^{\mathcal{G}} \parallel \sigma_i^{\mathcal{G}}], \quad \mu_i^{\mathcal{G}}, \sigma_i^{\mathcal{G}} \in \mathbb{R}^D \quad (12)$$

$$\hat{\mathbf{X}}_i^{(\mathcal{G},U)} = \mu_i^{\mathcal{G}} + \sigma_i^{\mathcal{G}} \times \epsilon \quad (13)$$

where $\mu_i^{\mathcal{G}}$ is the mean vector representation of exogenous variable $U_i^{\mathcal{G}}$; $\sigma_i^{\mathcal{G}}$ represents the standard deviation vector representation of exogenous variable $U_i^{\mathcal{G}}$; D denotes the latent dimension; ϵ is a value sampled from the normal distribution.

SCM Decoder aims to approximate the causal function $\mathcal{F}_i$, thereby reconstructing the endogenous variables based on samples. The specific operations are similar to SCM Encoder. The difference is that the input is $\hat{\mathbf{X}}_i^{(\mathcal{G},U)}$ and the output is $\hat{\mathbf{X}}_i^{(\mathcal{G},V)}$ when approximating $\mathcal{F}_i$.

SCM Loss aims to ensure accurate learning of causal relationships. Because CVGA focuses more on various variables rather than causal structure in SCMs, it directly uses the loss function of VAE to guide model training. SCM Loss is given by the following equation.

$$\mathcal{L} = \sum_{\mathcal{G} \in \mathcal{M}_n} \sum_{i \in V^{\mathcal{G}}} \frac{\text{MSE}\left(\mathbf{X}_i^{(\mathcal{G},V)}, \hat{\mathbf{X}}_i^{(\mathcal{G},V)}\right) + \text{KL}\left(\mathcal{N}(\mu_i^{\mathcal{G}}, (\sigma_i^{\mathcal{G}})^2), \mathcal{N}(0, 1)\right)}{|\mathcal{M}_n| \times |V^{\mathcal{G}}|} \quad (14)$$

where $\mathcal{L}$ is the loss value; $\mathcal{M}_n$ represents the set of normal SCMs; $V^{\mathcal{G}}$ denotes the endogenous variable set of SCM $\mathcal{G}$; $\text{MSE}(\cdot, \cdot)$ represents the mean-square error; $\text{KL}(\cdot, \cdot)$ is the Kullback-Leibler divergence.

3.1.4 Gaussian Mixture Distribution-based Anomaly Detector. After getting the vector representations of exogenous variables in SCMs, we need to design an anomaly detector that can provide accurate normal probabilities of microservice systems. Gaussian Mixture Distribution is a probability distribution model that approximates the probability distribution by combining multiple Gaussian distributions. It can adapt to various data distributions and give probabilistic explanations for samples. Therefore, a Gaussian Mixture Distribution-based Anomaly Detector (GAD) is proposed to detect anomalies.

An SCM can be represented by only its exogenous variables since each endogenous variable can be expressed as a composite function of exogenous variables. Besides, there are a large number of exogenous variables in an SCM. Therefore, the global mean pooling is used to calculate the vector representations of SCMs. The specific pooling process is shown by the following equation.

$$\mathbf{H}^{\mathcal{G}} = \frac{1}{|U^{\mathcal{G}}|} \sum_{i \in U^{\mathcal{G}}} \mathbf{x}_i^{(g,u)} \quad (15)$$

where $\mathbf{H}^{\mathcal{G}}$ is the vector representation of SCM $\mathcal{G}$; $U^{\mathcal{G}}$ represents the exogenous variable set of SCM $\mathcal{G}$. Then, Gaussian Mixture Distribution is used to model the vector representations of SCMs in the fault-free phase, which is given by the following equation.

$$\mathbf{P}(\mathbf{H}^{\mathcal{G}}|\Theta) = \sum_{i=1}^D \omega_i \times \mathcal{N}(\mathbf{H}^{\mathcal{G}}|\mu_i, \Sigma_i) \quad (16)$$

where Θ is the parameter set of Gaussian Mixture Distribution $\mathbf{P}(\mathbf{H}^{\mathcal{G}}|\Theta)$; ω_i denotes the weight of the i -th Gaussian component; $\mathcal{N}(\mathbf{H}^{\mathcal{G}}|\mu_i, \Sigma_i)$ is the probability density function of the i -th Gaussian component. Afterwards, the Gaussian Mixture Distribution $\mathbf{P}(\mathbf{H}^{\mathcal{G}}|\Theta)$ is learned by the Expectation-Maximization algorithm. Finally, we use the Gaussian Mixture Distribution $\mathbf{P}(\mathbf{H}^{\mathcal{G}}|\Theta)$ to calculate the log-likelihood value of an SCM, and use this value as the normal probability of a microservice system.

To distinguish abnormal SCMs, a threshold δ is determined based on the log-likelihood values of SCMs in the fault-free phase and an SCM whose log-likelihood value is lower than δ is regarded as abnormal, as defined below:

$$\delta = \text{Percentile} \left(\{\log \mathbf{P}(\mathbf{H}^{\mathcal{G}}|\Theta)\}_{\mathcal{G} \in \mathcal{M}_n}, \varepsilon \right) \quad (17)$$

where ε is the desired false positive rate (e.g., 0.5%, 1% or 1.5%), and $\mathcal{M}_n$ denotes the set of normal SCMs.

3.2 Counterfactual Reasoning-based Root Cause Locator

In the fault-suffering phase, Counterfactual Reasoning-based Root Cause Locator (CRCL) will be triggered when GAD detects a fault. It will localize root causes and provide effective guidance for subsequent troubleshooting. Counterfactual reasoning is a cognitive process that investigates how the results would be different if a past event or behavior had changed [9, 28, 30]. In microservice systems, root cause localization is essentially a counterfactual attribution problem. Therefore, Counterfactual reasoning is suitable for root cause localization of microservice systems. More importantly, it can tell operators why root causes are culprits and how to deal with them to effectively mitigate faults.

The Abduction-Action-Prediction process customized to microservice systems is designed to localize root causes and provide effective guidance for fault mitigation. This process consists of three steps: **CVGA-based Abduction**, **NDPL-based Intervention**, and **CVGA-GAD-based Prediction**.

3.2.1 CVGA-based Abduction. In Counterfactual Reasoning-based Root Cause Locator, CVGA-based Abduction is the basic component, which aims to infer the potential causes of the observed data.

In an SCM, each endogenous variable can be expressed as a composite function of exogenous variables. Therefore, CVGA-based Abduction uses CVGA to calculate exogenous variables' vector representations, and treats them as the cause representations of the observed data. The specific Abduction process is shown by the following equation.

$$\mathbf{X}^{(g,u)} = f(\mathbf{X}^{(g,v)}, \mathbf{A}^{(g,v)}) \quad (18)$$

where $\mathbf{X}^{(g,u)}$ and $\mathbf{X}^{(g,v)}$ represent the vector representations of exogenous variables and endogenous variables in SCM $\mathcal{G}$, respectively; $\mathbf{A}^{(g,v)}$ is the matrix of causal relationships between endogenous variables in SCM $\mathcal{G}$; $f(\cdot)$ denotes the SCM Encoder in CVGA.

3.2.2 NDPL-based Intervention. In Counterfactual Reasoning-based Root Cause Locator, NDPL-based Intervention is the core component, which aims to perform external operation that enforces causal assumptions under the constraints of SCMs to generate intervention SCMs that reflect different counterfactual assumptions. Although NDPL-based Intervention rely on predefined normal patterns, they are not simple pattern-matching or replacing mechanisms. NDPL is a structured library of typical causal behavior patterns learned from multi-source observability data. These causal behavior patterns can provide a reasonable reference for counterfactual interventions. Each intervention operation is generated under the causal constraints of SCMs, which can ensure the effectiveness and interpretability of the generated intervention SCMs.

In an SCM, there are a large number of events, each with a large number of intervention operations. To ensure the feasibility and existence of intervention operations in real microservice systems, the customized intervention operations for each event are designed based on NDPL to generate intervention SCMs that reflect different counterfactual assumptions.

For a non-Metric Event V_i^g , there are many related event patterns in NDPL, any of which could potentially be the correct event pattern. Therefore, NDPL-based Intervention first finds all related event patterns from NDPL according to the predecessor non-Metric Events of V_i^g . Then, an intervention operation set is generated based on these event patterns, where each intervention operation corresponds to a related event pattern. The specific generation rule is given by the following equation.

$$\mathcal{I}_i^g = \{ do(V_i^g = V_k^g) \mid (V_j^g, V_i^g) \in \mathcal{P}_E, (V_j^g, V_k^g) \in \mathcal{P}_E, 1 \leq j, k \leq |V^g|, i \neq j \neq k \} \quad (19)$$

where $\mathcal{I}_i^g$ represents the intervention operation set for V_i^g ; V^g denotes the endogenous variable set of SCM $\mathcal{G}$.

For a Local Metric Event V_i^g , it describes the local resource status of a service instance within a time interval. Therefore, all Local Metric Events about this service instance within this time interval require intervention. NDPL-based Intervention first finds all related local metric patterns from NDPL according to the service instance $\phi(V_i^g)$ and resource type $\psi(V_i^g)$. Then, an intervention operation set is generated based on the related local metric patterns, where each intervention operation corresponds to a Local Metric Event about this service instance within this time interval. The specific generation rule is shown by the following equation.

$$\mathcal{I}_i^g = \{ do(V_j^g = \mathcal{P}_L(\phi(V_i^g), \psi(V_i^g), mode)) \mid \phi(V_j^g) = \phi(V_i^g), \psi(V_j^g) = \psi(V_i^g), 1 \leq j \leq |V^g| \} \quad (20)$$

where $\mathcal{I}_i^g$ represents the intervention operation set for V_i^g .

For a Transmission Metric Event V_i^g , it describes the transmission resource status between the caller service instance and callee service instance within a time interval. This status can be affected by either of both service instances. Therefore, NDPL-based Intervention requires determining which service instance is suffering from faults and designing intervention operations for them. The related Transmission Metric Events are divided into

caller events and callee events. The specific division rules are shown by the following equations.

$$\begin{cases} E_{pre,out}^{(g,i)} &= \{ V_j^g \mid \psi(V_i^g) = \psi(V_j^g), \rho_1(V_i^g) = \rho_1(V_j^g), 1 \leq j \leq |V^g| \} \\ E_{pre,in}^{(g,i)} &= \{ V_j^g \mid \psi(V_i^g) = \psi(V_j^g), \rho_1(V_i^g) = \rho_2(V_j^g), 1 \leq j \leq |V^g| \} \\ E_{pos,out}^{(g,i)} &= \{ V_j^g \mid \psi(V_i^g) = \psi(V_j^g), \rho_2(V_i^g) = \rho_1(V_j^g), 1 \leq j \leq |V^g| \} \\ E_{pos,in}^{(g,i)} &= \{ V_j^g \mid \psi(V_i^g) = \psi(V_j^g), \rho_2(V_i^g) = \rho_2(V_j^g), 1 \leq j \leq |V^g| \} \end{cases} \quad (21)$$

where $E_{pre,out}^{(g,i)}$ is the caller events of the caller service instance of V_i^g ; $E_{pre,in}^{(g,i)}$ represents the callee events of the caller service instance of V_i^g ; $E_{pos,out}^{(g,i)}$ denotes the caller events of the callee service instance of V_i^g ; $E_{pos,in}^{(g,i)}$ is the callee events of the callee service instance of V_i^g ; $\rho_1(V_i^g)$ and $\rho_2(V_i^g)$ denote the caller service instance and callee service instance of V_i^g , respectively. Then, it finds all related transmission metric patterns from NDPL according to two service instances and resource type. Afterwards, three candidate sets of intervention operations are generated based on the related transmission metric patterns and abnormal service instances, where each intervention operation corresponds to a Transmission Metric Event about the abnormal service instance within this time interval. The specific generation rules are given by the following equations.

$$\mathcal{J}_{i,pre}^g = \{ do(V_j^g = \mathcal{P}_L(\rho_1(V_j^g), \rho_2(V_j^g), \psi(V_j^g), mode)) \mid V_j^g \in E_{pre,out}^{(g,i)} \cup E_{pre,in}^{(g,i)} \} \quad (22)$$

$$\mathcal{J}_{i,pos}^g = \{ do(V_j^g = \mathcal{P}_L(\rho_1(V_j^g), \rho_2(V_j^g), \psi(V_j^g), mode)) \mid V_j^g \in E_{pos,out}^{(g,i)} \cup E_{pos,in}^{(g,i)} \} \quad (23)$$

$$\mathcal{J}_{i,oth}^g = \begin{cases} \mathcal{J}_{i,pre}^g, & \text{if } |E_{pre,out}^{(g,i)} \cup E_{pre,in}^{(g,i)}| > |E_{pos,out}^{(g,i)} \cup E_{pos,in}^{(g,i)}|, \\ \mathcal{J}_{i,pos}^g, & \text{otherwise.} \end{cases} \quad (24)$$

where $\mathcal{J}_{i,pre}^g$, $\mathcal{J}_{i,pos}^g$ and $\mathcal{J}_{i,oth}^g$ are three candidate sets of intervention operations. Finally, a candidate set of invention operations is chosen based on different performance of service instances. The specific selection rules are given by the following equation.

$$\mathcal{J}_i^g = \begin{cases} \mathcal{J}_{i,pre}^g, & \text{if } \forall V_j^g \in E_{pre,out}^{(g,i)} \cup E_{pre,in}^{(g,i)}, V_j^g > \mathcal{P}_L(\rho_1(V_j^g), \rho_2(V_j^g), \psi(V_j^g), max) \\ \mathcal{J}_{i,pos}^g, & \text{if } \forall V_j^g \in E_{pos,out}^{(g,i)} \cup E_{pos,in}^{(g,i)}, V_j^g > \mathcal{P}_L(\rho_1(V_j^g), \rho_2(V_j^g), \psi(V_j^g), max) \\ \mathcal{J}_{i,oth}^g, & \text{otherwise.} \end{cases} \quad (25)$$

where $\mathcal{J}_i^g$ represents the intervention operation set for V_i^g .

Once the intervention operation set for each event has been determined, the next step is to apply these intervention operations to the corresponding events. Through this operation, a series of intervention SCMs will be created, each of which reflects a specific counterfactual assumption. Because variables are updated synchronously in CVGA and CVGA only has a convolutional layer, intervention operations only require changing the values of intervened variables without removing non-causal information flows. The specific generation rule is shown by the following equation.

$$\mathcal{Q}_i^g = \{ \Lambda(\mathcal{G}) \mid \Lambda \in \mathcal{J}_i^g \}, i = 1, \dots, |V^g| \quad (26)$$

where $\mathcal{Q}_i^g$ represents the intervention SCM set of the endogenous variable V_i^g in SCM $\mathcal{G}$; V^g denotes the endogenous variable set of SCM $\mathcal{G}$. Fig. 3 shows two examples of interventions on the SCMs in Fig. 2, where red prompt boxes represent intervention operations.

After getting intervention SCMs for each event, an intervention SCM needs to be described by a vector representation. In an SCM, each endogenous variable can be expressed as a composite function of exogenous variables. Thus, we can describe an intervention SCM by its exogenous variables. Specifically, CVGA is used to

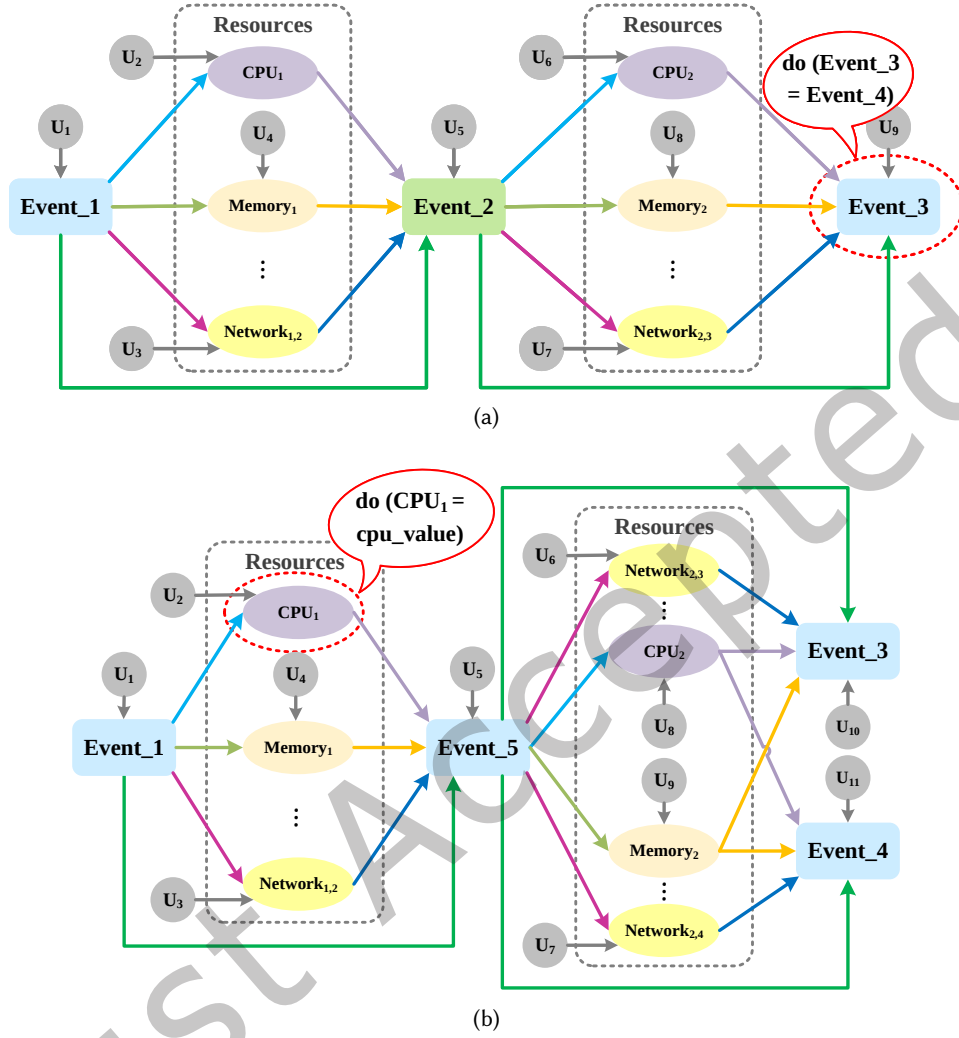


Fig. 3. Two examples of interventions on the SCMs in Fig. 2.

calculate the vector representations of the exogenous variables in an intervention SCM. The specific process is shown by the following equation.

$$\mathbf{X}^{(Q_{i,j}^g, U)} = f(\mathbf{X}^{(Q_{i,j}^g, V)}, \mathbf{A}^{(Q_{i,j}^g, V)}) \quad (27)$$

where $\mathbf{X}^{(Q_{i,j}^g, U)}$ and $\mathbf{X}^{(Q_{i,j}^g, V)}$ represent the feature map of the exogenous and endogenous variables in the j -th intervention SCM of V_i^g , respectively; $f(\cdot)$ is the SCM Encoder in CVGA.

3.2.3 CVGA-GAD-based Prediction. In Counterfactual Reasoning-based Root Cause Locator, CVGA-GAD-based Prediction is the key component, which aims to predict counterfactual outcomes based on counterfactual assumptions.

Counterfactual assumptions contain the updated knowledge and intervention SCMs. Among them, the updated knowledge is described by the cause representations inferred by CVGA-based Abduction, and intervention SCMs are described by their exogenous variables' vector representations calculated by NDPL-based Intervention.

In an SCM, each endogenous variable can be expressed by a composite function of a set of exogenous variables. Therefore, the change of an endogenous variable must be caused by the change of a set of exogenous variables. To reflect the inhere rule in counterfactual assumptions, CVGA-GAD-based Prediction first finds the exogenous variables not affected by the intervention operation from SCM $\mathcal{G}$. Then, it replaces their vector representations with the corresponding cause representations to obtain the vector representations of all exogenous variables in counterfactual assumptions. The specific generation rule is given by the following equations.

$$\mathbf{Z}_{i,j,k}^{(g,u)} = \begin{cases} \mathbf{X}_k^{(g,u)}, & \text{if } k \notin \{i \mid V_i^g \in Ch(V_i^g)\} \cup \{i\}, \\ \mathbf{X}_k^{(Q_{i,j}^g, U)}, & \text{otherwise.} \end{cases} \quad (28)$$

where $\mathbf{Z}_{i,j,k}^{(g,u)}$ is the vector representation of exogenous variable U_k^g in the j -th counterfactual assumption of V_i^g ; $\mathbf{X}_k^{(Q_{i,j}^g, U)}$ represents the vector representation of exogenous variable U_k^g in the j -th intervention SCM of V_i^g ; $\mathbf{X}_k^{(g,u)}$ denotes the cause representation of exogenous variable U_k^g ; $Ch(V_i^g)$ is the set of child endogenous variables of V_i^g . Fig. 4 shows two examples of counterfactual assumptions on the SCMs in Fig. 2. In Fig. 4, red circles represent exogenous variables affected by an intervention operation, gray circles denote exogenous variables not affected by an intervention operation, and red prompt boxes represent intervention operations.

Then, the vector representations of counterfactual assumptions are calculated. In a counterfactual assumption, there are a large number of exogenous variables. These exogenous variables describe counterfactual assumptions from different perspectives. Thus, the global mean pooling is used to calculate the vector representations of counterfactual assumptions. The specific pooling process is shown by the following equation.

$$\hat{\mathbf{H}}_{i,j}^g = \frac{1}{|U^g|} \sum_{k \in U^g} \mathbf{Z}_{i,j,k}^{(g,u)} \quad (29)$$

where $\hat{\mathbf{H}}_{i,j}^g$ represents the vector representation of the j -th counterfactual assumption of V_i^g .

After that, the counterfactual outcomes are predicted. In the fault diagnosis of microservice systems, anomaly detection and root cause localization are two closely related phases. To achieve effective feedback and coordination between both phases, GAD is used to predict counterfactual outcomes. The specific prediction rule is given by the following equation.

$$Score_{i,j}^g = g(\hat{\mathbf{H}}_{i,j}^g) \quad (30)$$

where $Score_{i,j}^g$ is the score for the j -th counterfactual assumption of V_i^g ; $g(\cdot)$ denotes GAD.

Finally, all events are ranked by their best counterfactual outcomes. The higher the ranking, the more likely the event is a root cause, and its corresponding intervention operation is the corresponding measure of fault mitigation. The specific rules are given by the following equations.

$$\begin{cases} j^* = \arg \max_{1 \leq j \leq |Q_i^g|} Score_{i,j}^g \\ C_i^g = rank(\mathcal{J}_{i,j^*}^g, Score_{i,j^*}^g) \end{cases} \quad (31)$$

where C_i^g represents the ranking of V_i^g ; $rank(\cdot, \cdot)$ denotes the ranking function. The above ranking process not only identifies the most likely root cause but also provides potential intervention operations that may facilitate the recovery of microservice systems. These intervention operations are actionable recommendations derived through counterfactual reasoning, aiming to provide operators with interpretable decision support for fault

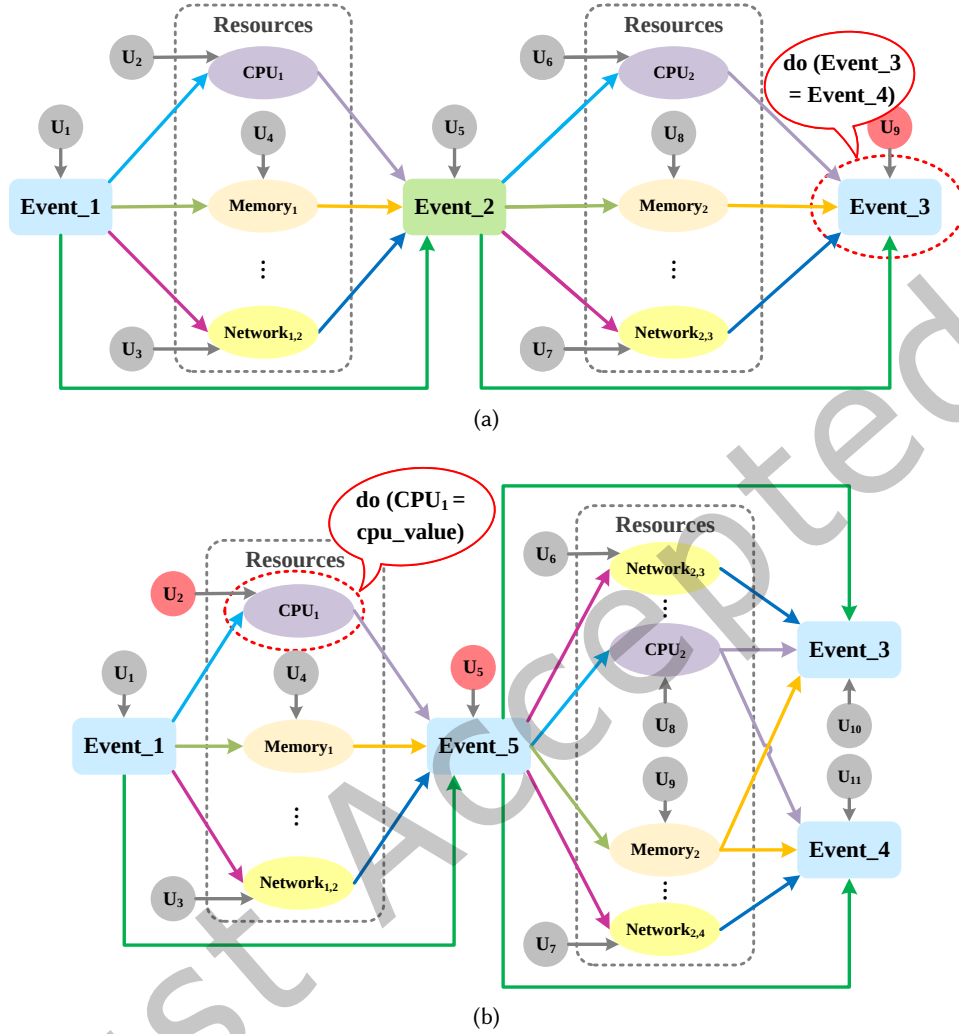


Fig. 4. Two examples of counterfactual assumptions on the SCMs in Fig. 2.

mitigation rather than universally effective remedies. For example, in a real case from the TT dataset, CRFD detects a fault and triggers the root cause locator, identifying a CPU-related event in ts-order-service as the most likely root cause. Based on counterfactual analysis, one possible intervention is to reduce CPU utilization to 20%. This result indicates that the high CPU utilization of ts-order-service is highly correlated with the current fault, and reducing CPU utilization to 20% may help mitigate this fault. It is worth noting that such intervention operations may introduce additional side effects and their effectiveness depends on the system configuration and workload characteristics. Although these intervention operations are not physically executed in this study, their interpretability and plausibility are supported by counterfactual reasoning. Therefore, they should be regarded

as decision-support recommendations that help operators in fault diagnosis and exploring feasible mitigation strategies, rather than prescriptive actions that are universally applicable.

4 Evaluation

CRFD is implemented using Python 3.7, PyTorch 1.11.0 and PyTorch Geometric 2.1.0. Based on this implementation, we conduct extensive experiments to answer the following questions.

- **RQ1:** How effective is CRFD in anomaly detection compared with baselines?
- **RQ2:** How effective is CRFD in service-level root cause localization compared with baselines?
- **RQ3:** How effective is CRFD in inner-service-level root cause localization compared with baselines?
- **RQ4:** How efficient is CRFD in anomaly detection compared with baselines?
- **RQ5:** How efficient is CRFD in root cause localization compared with baselines?
- **RQ6:** How much does CRCL contribute to CRFD?
- **RQ7:** How much does multi-source observability data contribute to CRFD?
- **RQ8:** How does the scalability of CRFD change with trace length and system size?
- **RQ9:** How do different hyperparameters affect the effectiveness of CRFD?

4.1 Experimental Setup

4.1.1 Datasets. Experiments are conducted on two datasets collected from the Train Ticket microservice system [13, 31, 37, 39, 40, 45] and the Social Network microservice system. The details of datasets are given in Table 3.

Table 3. Details of Datasets.

Datasets	Instances	Normal Traces	Abnormal Traces	Anomaly Cases
TT	45	18400	1736	CPU Exhaustion
				Network Jam
				Code Defect
SN	21	13612	1106	CPU Exhaustion
				Network Jam
				Network Loss

- **TT.** This dataset is collected from the Train Ticket microservice system with 45 microservices [13, 31, 37, 39, 40, 45]. We deploy this microservice system on a Kubernetes platform with 6 machines, each of which has a 4-core 3.60GHz CPU and 8GB memory, and runs with CentOS 7.6 OS. We use Apache SkyWalking [29] to collect traces and logs, Prometheus [24] to collect metrics, and Elasticsearch [5] to store the collected data. We mimic three types of faults: CPU Exhaustion, Network Jam and Code Defect. For CPU Exhaustion and Network Jam, we mimic them by fault injection following the previous works [15, 17, 36]. For Code Defect, we mimic it by modifying the source code. Finally, 18400 normal traces and 1736 abnormal traces were collected.
- **SN.** This dataset is provided by [13] and collected from the Social Network microservice system with 21 microservices [6]. It contains three types of faults: CPU Exhaustion, Network Jam and Network Loss. In experiments, 13612 normal traces and 1106 abnormal traces were used.

4.1.2 Baselines. CRFD is an integrated approach for the fault diagnosis of microservice systems, which can simultaneously complete anomaly detection and root cause localization. To evaluate the performance of CRFD in terms of anomaly detection, five state-of-the-art anomaly detection approaches are used as baselines.

- **DeepLog** [4]. A log-based anomaly detection approach. This approach uses LSTM to learn the sequential patterns of logs, and detect anomalies by predicting the next log event.
- **LogAnomaly** [20]. A log-based anomaly detection approach. This approach uses LSTM to learn the sequential and quantitative patterns of logs, and detect anomalies by predicting the next log event.
- **TraceAnomaly** [17]. A trace-based anomaly detection approach. This approach combines STV [34] with VAE to learn the invocation patterns of traces, and detect anomalies by identifying outlier traces.
- **DeepTraLog** [39]. A multi-source-data-based anomaly detection approach. This approach combines GGNNs with Deep SVDD to learn the behavior patterns of traces and logs, and detect anomalies by identifying outlier behaviors.
- **iTCRL-LOF** [31]. A multi-source-data-based anomaly detection approach. This approach combines iTCRL with the LOF algorithm to learn the behavior patterns of traces and logs, and detect anomalies by identifying outlier behaviors.

To evaluate the performance of CRFD in terms of root cause localization, seven state-of-the-art root cause localization approaches are used as baselines.

- **TBAC** [19]. A metric-based root cause localization approach. This approach combines response time with the dependency relation graph to localize root cause services.
- **MonitorRank** [12]. A metric-based root cause localization approach. This approach uses a random walk to localize root cause services.
- **CloudRanger** [32]. A metric-based root cause localization approach. This approach combines the PC algorithm [30] with the second-order random walk to localize root cause services.
- **TraceAnomaly** [17]. A trace-based root cause localization approach. This approach compares call paths in the fault-free phase with those in the fault-suffering phase to localize root cause services.
- **MicroRank** [36]. A trace-based root cause localization approach. This approach combines the extended spectrum techniques with PageRank to localize root cause services.
- **CausalRCA** [34]. A metric-based root cause localization approach. This approach combines the gradient-based causal structure learning method with PageRank to localize root cause metrics.
- **Nezha** [37]. A multi-source-data-based root cause localization approach. This approach compares event patterns in the fault-free phase with those in the fault-suffering phase to localize root cause resources or code regions.

4.1.3 Evaluation Metrics. For the anomaly detection task, the following three metrics are used to evaluate the effectiveness of all approaches based on TP (True Positive), FP (False Positive) and FN (False Negative).

- **Precision** refers to the proportion of samples that are actually abnormal among all samples detected as anomalies. The specific calculation equation is shown as follows.

$$\text{Pre} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (32)$$

- **Recall** refers to the proportion of samples that are correctly detected as anomalies among all samples that are actually anomalies. The specific calculation equation is given as follows.

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (33)$$

- **F1-Score** refers to the harmonic mean of Precision and Recall. The specific calculation equation is shown as follows.

$$F1 = \frac{2 \times \text{Pre} \times \text{Rec}}{\text{Pre} + \text{Rec}} \quad (34)$$

For the root cause localization task, the following four metrics are used to evaluate the effectiveness of all approaches.

- **AS@k** refers to the probability that root cause services appear in the top-k localization results. The specific calculation equation is given as follows.

$$\text{AS}@k = \frac{1}{|\mathcal{M}_a|} \sum_{\mathcal{G} \in \mathcal{M}_a} \frac{\sum_{i=1}^k \mathbb{1}_{CS_i^{\mathcal{G}} \in RS_{\mathcal{G}}}}{\min(k, |RS_{\mathcal{G}}|)} \quad (35)$$

where $\mathcal{M}_a$ is the set of abnormal SCMs; $CS_i^{\mathcal{G}}$ represents the i -th service instance in the localization results of the sample $\mathcal{G}$; $RS_{\mathcal{G}}$ denotes the root cause service set of the sample $\mathcal{G}$.

- **AvgS@k** refers to the mean probability that root cause services appear in the top-k localization results. The specific calculation equation is shown as follows.

$$\text{AvgS}@k = \frac{1}{k} \sum_{i=1}^k \text{AS}@i \quad (36)$$

- **AIS@k** refers to the probability that the inner-service root causes appear in the top-k localization results. The specific calculation equation is given as follows.

$$\text{AIS}@k = \frac{1}{|\mathcal{M}_a|} \sum_{\mathcal{G} \in \mathcal{M}_a} \frac{\sum_{i=1}^k \mathbb{1}_{CE_i^{\mathcal{G}} \in RE_{\mathcal{G}}}}{\min(k, |RE_{\mathcal{G}}|)} \quad (37)$$

where $CE_i^{\mathcal{G}}$ represents the i -th inner-service root cause in the localization results of the sample $\mathcal{G}$; $RE_{\mathcal{G}}$ denotes the inner-service root cause set of the sample $\mathcal{G}$.

- **AvgI@k** refers to the mean probability that the inner-service root causes appear in the top-k localization results. The specific calculation equation is shown as follows.

$$\text{AvgI}@k = \frac{1}{k} \sum_{i=1}^k \text{AIS}@i \quad (38)$$

4.1.4 Settings. All experiments are conducted on a server with an Intel Core i7-12700 2.10GHz CPU, 64GB RAM, RTX 4080 with 16GB GPU memory. The settings of CRFD are as follows: we use the Adaptive Moment Estimation (Adam) optimizer with a Cosine Annealing Learning Rate Scheduler. The batch size is set to 128, the number of epochs is set to 15, the latent dimension D is set to 16, the initial learning rate η is set to 0.001, and the desired false positive rate ε is set to 0.5. In our experiments, normal traces are divided into the training, validation, and test sets in the ratio of 8 : 1 : 1. For abnormal traces, 30% are randomly sampled and put in the validation set for model tuning and hyperparameter sensitivity analysis, while the remaining 70% are put in the test set for performance evaluation. To mitigate the effect of randomness, all experiments are repeated five times, and the average results are reported. The source code and datasets of CRFD are available at <https://github.com/intse/CRFD>.

4.2 Effectiveness of Anomaly Detection (RQ1)

Based on the known injected operations, the current trace is abnormal if a fault is injected; otherwise, it is normal. Table 4 shows the effectiveness evaluation results of different approaches in terms of anomaly detection.

Table 4. Effectiveness evaluation results for anomaly detection.

Approach	TT			SN		
	Pre	Rec	F1	Pre	Rec	F1
DeepLog	0.4652	0.7431	0.5722	0.4945	0.9015	0.6387
LogAnomaly	0.4639	0.7437	0.5714	0.4968	0.8987	0.6399
TraceAnomaly	0.4912	0.9614	0.6502	0.6605	0.7423	0.6990
DeepTraLog	0.4855	1.0000	0.6536	0.4483	1.0000	0.6191
iTCRL-LOF	0.9110	0.9793	0.9439	0.8792	1.0000	0.9357
CRFD	0.9910	0.9992	0.9951	0.9843	0.9106	0.9448

As shown in Table 4, CRFD achieves the best overall performance in terms of Pre (0.9877), Rec (0.9549) and F1 (0.9700) on average. This is due to the SCM-based aggregation of multi-source observability data so that it can capture causal relationships among events across metrics, traces and logs.

The two log-based anomaly detection approaches DeepLog and LogAnomaly obtain an F1 of around 0.6. Their average F1 reach almost 0.6055 and 0.6057, respectively. Both approaches only consider logs. Thus, they are unable to detect resource issues that cannot cause a change in log sequences.

The trace-based anomaly detection approach TraceAnomaly achieves a low F1. Its average F1 reaches almost 0.6746. This approach only takes response time and call paths into account. Thus, it misses anomalies caused by code defects or resource issues.

The multi-source-data-based anomaly detection approach iTCRL-LOF achieves better overall performance than single-source-data-based approaches, while DeepTraLog achieves better Rec than single-source-data-based approaches. However, they are still worse than CRFD. This is because both approaches only consider traces and logs. Thus, they cannot detect anomalies only reflected in system-level metrics.

In brief, CRFD increases Pre by 8.78% ~ 119.56% and F1 by 0.97% ~ 74.15% than baselines, while it is slightly worse than iTCRL-LOF and DeepTraLog in terms of Rec. CRFD detects anomalies by probabilistic distribution modeling on SCM representations. This high-confidence detection strategy can suppress false positives and miss some weak anomalies, which leads to a slightly lower Rec. Overall, this trade-off improves the Pre and reliability of anomaly detection, which can provide more trustworthy inputs for subsequent root cause localization.

4.3 Effectiveness of Root Cause Localization at Service Level (RQ2)

The ground truths at service level are the known injected services. Table 5 shows the effectiveness evaluation results of different approaches in terms of root cause localization at service level.

From Table 5, we can observe that CRFD significantly outperforms all baselines and achieves the highest accuracy in terms of AS@1 (0.9316), AS@3 (0.9893), AS@5 (0.9959) and AvgS@5 (0.9784) on average. The excellent performance of CRFD is due to the SCM-based aggregation of multi-source observability and the use of counterfactual reasoning. The former allows CRFD to capture causal relationships among multi-source observability data, and the latter can enable fine-grained and interpretable root cause localization by simulating scenarios where no fault occurs.

The four metric-based root cause localization approaches TBAC, MonitorRank, CloudRanger and CausalRCA achieve poor performance. This is because these approaches only consider metrics, which makes them unable to localize faults caused by invocation anomalies and code defects.

The two trace-based root cause localization approaches TraceAnomaly and MicroRank obtain AS@1 and AvgS@5 less than 0.5. TraceAnomaly takes response time and call paths into account, while MicroRank only considers call paths. Thus, they cannot localize faults caused by resource issues and code defects.

Table 5. Effectiveness evaluation results for root cause localization at service level.

Approach	TT				SN			
	AS@1	AS@3	AS@5	AvgS@5	AS@1	AS@3	AS@5	AvgS@5
TBAC	0.5882	0.5882	0.6176	0.5941	0.0896	0.2751	0.4606	0.2793
MonitorRank	0.2843	0.8725	0.8824	0.7275	0.1459	0.3053	0.4646	0.3037
CloudRanger	0.4412	0.8235	0.8824	0.7569	0.0981	0.2527	0.4424	0.2687
TraceAnomaly	0.1578	0.4251	0.4251	0.3658	0.0651	0.2740	0.2939	0.2244
MicroRank	0.0305	0.6475	0.6475	0.4404	0.3083	0.4801	0.6139	0.4584
CausalRCA	0.0294	0.1667	0.2451	0.1529	0.4019	0.7068	0.7825	0.6514
Nezha	0.6757	0.9424	0.9850	0.8992	0.9195	0.9684	0.9684	0.9542
CRFD	0.9254	0.9849	0.9970	0.9748	0.9377	0.9938	0.9948	0.9819

The multi-source-data-based root cause localization approach Nezha achieves better performance than single-source-data-based approaches. Its average AS@1, AS@3, AS@5 and AvgS@5 reach almost 0.7976, 0.9554, 0.9767 and 0.9267, respectively. However, it is still worse than CFRD. This approach treats system-level metrics as the highest localization priority without investigating their severities and the causes behind resource issues. Thus, the first one will be output when multiple behaviors do not conform to normal event patterns.

In conclusion, CRFD outperforms all baselines, achieving performance improvements ranging from 1.98% to 3046.47% in AS@1, 2.63% to 490.91% in AS@3, 1.22% to 306.79% in AS@5, and 2.90% to 537.38% in AvgS@5. The evaluation results validate the effectiveness of CRFD in terms of root cause localization at service level.

4.4 Effectiveness of Root Cause Localization at Inner-Service Level (RQ3)

The ground truths at inner-service level are events resulting from the known injected operations. We only compare CRFD with CausalRCA and Nezha since only these three approaches can localize root cause at inner-service level. Table 6 shows the effectiveness evaluation results of different approaches in terms of root cause localization at inner-service level.

Table 6. Effectiveness evaluation results for root cause localization at inner-service level.

Approach	TT				SN			
	AIS@1	AIS@3	AIS@5	AvgI@5	AIS@1	AIS@3	AIS@5	AvgI@5
CausalRCA	0.0294	0.1078	0.1176	0.0922	0.2910	0.4424	0.5256	0.4279
Nezha	0.6757	0.9424	0.9850	0.8992	0.9195	0.9684	0.9684	0.9542
CRFD	0.9254	0.9695	0.9695	0.9602	0.9305	0.9915	0.9925	0.9776

As shown in Table 6, CRFD significantly outperforms all baselines and achieves the highest accuracy in terms of AIS@1 (0.9280), AIS@3 (0.9805), AIS@5 (0.9810) and AvgI@5 (0.9689) on average. This improvement is likely due to the SCM-based aggregation of multi-source observability data and the use of counterfactual reasoning so that it can localize root causes in a more fine-grained and interpretable manner.

The metric-based root cause localization approach CausalRCA achieves the worst performance. Its AIS@1, AIS@3, AIS@5 and AvgI@5 are less than 0.6. This approach only considers metrics and is highly sensitive to data quality. Thus, it can only localize faults reflected on metrics and cannot work well when dealing with observability data with substantial noise.

The multi-source-data-based root cause localization approach Nezha achieves better accuracy than the single-source-data-based approach. However, it is still worse than CRFD. This approach treats system-level metrics as the highest localization priority without investigating their severities and the causes behind resource issues. Thus, it tends to output the first located abnormal behavior as the root cause when multiple behaviors do not conform to normal event patterns.

In summary, CRFD outperforms all baselines on most evaluation metrics across both datasets, improving AIS@1 by 1.19% ~ 3046.47%, AIS@3 by 2.39% ~ 799.03%, and AvgI@5 by 2.45% ~ 941.92%. Although its AIS@5 on the TT dataset is slightly lower than that of Nezha, CRFD still achieves the best overall performance. The evaluation results validate the effectiveness of CRFD in terms of root cause localization at inner-service level.

4.5 Efficiency of Anomaly Detection (RQ4)

The efficiency of model training and anomaly detection plays an important role in fault diagnosis in microservice systems. During the effectiveness evaluation experiments of CRFD, we record both the model training time and the average detection time per case of the anomaly detector. Table 7 shows the efficiency evaluation results for anomaly detection.

Table 7. Efficiency evaluation results for anomaly detection.

Approach	TT		SN	
	Model Training Time (Seconds)	Average Detecting Time (Seconds)	Model Training Time (Seconds)	Average Detecting Time (Seconds)
DeepLog	1909.00	0.0278	155.00	0.0027
LogAnomaly	6563.00	0.2475	1409.00	0.0180
TraceAnomaly	358.02	0.0007	306.27	0.0013
DeepTraLog	4959.98	0.0037	3359.55	0.0016
iTCRL-LOF	349.96	0.0045	232.65	0.0019
CRFD	310.12	0.0014	244.80	0.0018

From Table 7, we can find that the overall performance of CRFD is best in terms of model training efficiency and ranks second in terms of anomaly detection efficiency. It achieves high efficiency in terms of Model Training Time (277.46 seconds) and Average Detecting Time (0.0016 seconds) on average. The excellent performance of CRFD is attributed to the simple architecture of the anomaly detector, so that it can describe multi-source observability data with a smaller representation dimension.

The two log-based anomaly detection approaches DeepLog and LogAnomaly exhibit low efficiency in both model training and anomaly detection, especially on the TT dataset. Log-based anomaly detection approaches need to use a sliding window to generate a large number of subsequences for training and detection, which increases time cost. Besides, LogAnomaly needs to consider both log event sequence and log quantitative sequence, which further increases the training time and detection time. Consequently, their efficiency will significantly decrease with the length of log event sequence, and LogAnomaly will drop more sharply than DeepLog.

The trace-based anomaly detection approach TraceAnomaly obtains Model Training Time and Average Detecting Time below 360 and 0.0015 seconds, respectively. This is because this approach only considers traces and has a simpler model architecture.

The two multi-source-data-based anomaly detection approaches DeepTraLog and iTCRL-LOF achieve notably different efficiency across datasets. In terms of Model Training Time and Average Detecting Time, DeepTraLog ranks sixth and third, whereas iTCRL-LOF ranks second and fourth on average. This is because the architecture of the former is more complex than the latter and requires more iterations to converge than the latter.

To sum up, CRFD achieves the best overall efficiency with the shortest model training time and near-optimal detection time among baselines.

4.6 Efficiency of Root Cause Localization (RQ5)

The efficiency of root cause localization is essential for fault diagnosis and subsequent troubleshooting. In the effectiveness evaluation experiments of CRFD, we record the average localization time per case. Table 8 shows the efficiency evaluation results for root cause localization.

Table 8. Efficiency evaluation result for root cause localization.

Approach	TT (Seconds/Case)	SN (Seconds/Case)
TBAC	4.1182	7.0099
MonitorRank	5.2687	1.9757
CloudRanger	6.6801	7.2161
TraceAnomaly	<0.0001	<0.0001
MicroRank	0.0372	0.0330
CausalRCA	58.7572	40.7751
Nezha	0.3822	0.3533
CRFD	2.1270	2.4834

From Table 8, we can observe that CRFD obtains an average localization time of 2.1270 and 2.4834 seconds per case on the TT and SN datasets, respectively. Although CRFD is worse than TraceAnomaly, MicroRank and Nezha, its average localization time is still much smaller than the sampling interval of observability data.

The four metric-based root cause localization approaches TBAC, MonitorRank, CloudRanger and CausalRCA exhibit poor performance. They obtain the average localization times ranging from 4.1182 to 58.7572 seconds on the TT dataset and from 1.9757 to 40.7751 seconds on the SN dataset. This inefficiency stems from the need to build causal or call graphs based on metrics, which is time-consuming.

The two trace-based root cause localization approaches TraceAnomaly and MicroRank achieve the best performance and obtain the average localization time below 0.1 seconds. Both approaches directly learn normal call paths from traces and localize root cause services by comparing call paths. Therefore, they can avoid the time-consuming process of building causal or call graphs.

The multi-source-data-based root cause localization approach Nezha achieves good performance and obtains the average localization time below 0.4 seconds. This efficiency results from its rule-based localization mechanism, which saves a large amount of time spent on model inference.

In summary, although CRFD does not achieve the highest localization efficiency, its average localization time remains below 3 seconds per case, which is sufficient to meet the requirements of real-time root cause localization.

4.7 Ablation Study: Contribution of CRCL (RQ6)

In CRFD, Counterfactual Reasoning-based Root Cause Locator (CRCL) is the most important component. To evaluate the contribution of CRCL to CRFD, we derive two variants of CRFD by replacing CRCL with different root cause locators, which are illustrated as follows.

- CRFD w/ SL: This variant replaces CRCL with a score-based locator that sorts events by their anomaly scores.
- CRFD w/ PL: This variant replaces CRCL with a PageRank-based locator that takes anomaly scores as personalization values and ranks nodes by their final PageRank scores.

Based on the above variants, we conduct a comprehensive ablation study on two datasets. Table 9 shows the contribution of CRCL to CRFD.

Table 9. Contribution of CRCL.

Approach	Service-level Root Cause Localization				Inner-service-level Root Cause Localization			
	TT		SN		TT		SN	
	AS@1	AvgS@5	AS@1	AvgS@5	AIS@1	AvgI@5	AIS@1	AvgI@5
CRFD w/ SL	0.4785	0.7836	0.9093	0.9707	0.1116	0.3977	0.2093	0.4743
CRFD w/ PL	0.6604	0.8349	0.8708	0.9684	0.2963	0.4674	0.4904	0.7904
CRFD	0.9254	0.9748	0.9377	0.9819	0.9254	0.9602	0.9305	0.9776

As shown in Table 9, CRFD significantly outperforms all variants at both service-level and inner-service-level root cause localization. When CRCL is replaced by the score-based locator, the average drop across both datasets for four evaluation metrics (AS@1, AvgS@5, AIS@1 and AvgI@5) is 25.51%, 10.34%, 82.71% and 55.00%, respectively. Similarly, replacing CRCL by PageRank-based locator leads to average drops of 17.82%, 7.84%, 57.61% and 35.09%. Overall, these significant performance degradations clearly show the contribution of CRCL to CRFD. This is because CRFD uses counterfactual reasoning to identify root causes rather than relying on anomaly scores, enabling more accurate and fine-grained root cause localization.

4.8 Ablation Study: Contribution of Multi-Source Observability Data (RQ7)

To evaluate the contribution of multi-source observability data to CRFD, we derive six variants of CRFD by retaining or removing different types of observability data, which are given as follows:

- CRFD w/ M: only metric data are retained.
- CRFD w/ T: only trace data are retained.
- CRFD w/ L: only log data is retained.
- CRFD w/o M: metric data are removed.
- CRFD w/o T: trace data are removed.
- CRFD w/o L: log data are removed.

Based on these variants, we conduct a comprehensive ablation study on two datasets. Table 10 shows the contribution of multi-source observability data to CRFD.

From Table 10, we can find that CRFD achieves the best overall performance on both datasets. Specifically, using only a single-source observability data (CRFD w/ M, CRFD w/ T, and CRFD w/ L) leads to severe performance degradations of about 10.19% ~ 99.36% across all evaluation metrics and datasets. Similarly, removing metrics or traces (CRFD w/o M and CRFD w/o T) also causes a significant drop of approximately 2.36% ~ 88.36% across all evaluation metrics and datasets. An exception occurs on the SN dataset, where the F1 slightly increases after removing logs, likely because no faults in the SN dataset are reflected in logs, and log data may introduce redundant information. Overall, these significant performance degradations clearly show the contribution of multi-source observability data to CRFD. This is because different faults are reflected in different observability data and no single observability data alone can distinguish all faults. Therefore, integrating multi-source observability data is essential for accurate fault diagnosis.

Table 10. Contribution of multi-source observability data.

Approach	Anomaly Detection		Service-level Root Cause Localization				Inner-service-level Root Cause Localization			
	TT		TT		SN		TT		SN	
	F1	F1	AS@1	AvgS@5	AS@1	AvgS@5	AIS@1	AvgI@5	AIS@1	AvgI@5
CRFD w/ M	0.2697	0.7652	0.1044	0.4676	0.8289	0.8818	0.0059	0.1923	0.8279	0.8307
CRFD w/ T	0.8484	0.3885	0.6313	0.8399	0.2842	0.5095	0.3542	0.3542	0.1176	0.1913
CRFD w/ L	0.5662	0.1672	0.4211	0.6553	0.1979	0.4971	0.2170	0.2774	0.1165	0.3162
CRFD w/o M	0.8542	0.3851	0.8273	0.9518	0.2938	0.4946	0.7184	0.7184	0.1083	0.1631
CRFD w/o T	0.7808	0.8137	0.4196	0.5663	0.8501	0.8867	0.4097	0.4430	0.8292	0.8397
CRFD w/o L	0.8864	0.9531	0.7264	0.8649	0.9119	0.9789	0.5992	0.6123	0.9047	0.9752
CRFD	0.9951	0.9448	0.9254	0.9748	0.9377	0.9819	0.9254	0.9602	0.9305	0.9776

4.9 Scalability of CRFD (RQ8)

To evaluate the scalability of CRFD, we further investigate the time cost of its key stages (SCM Construction, CVGA Training, and CRCL Inference) under different trace lengths and system sizes, as shown in Fig. 5 and Fig. 6.

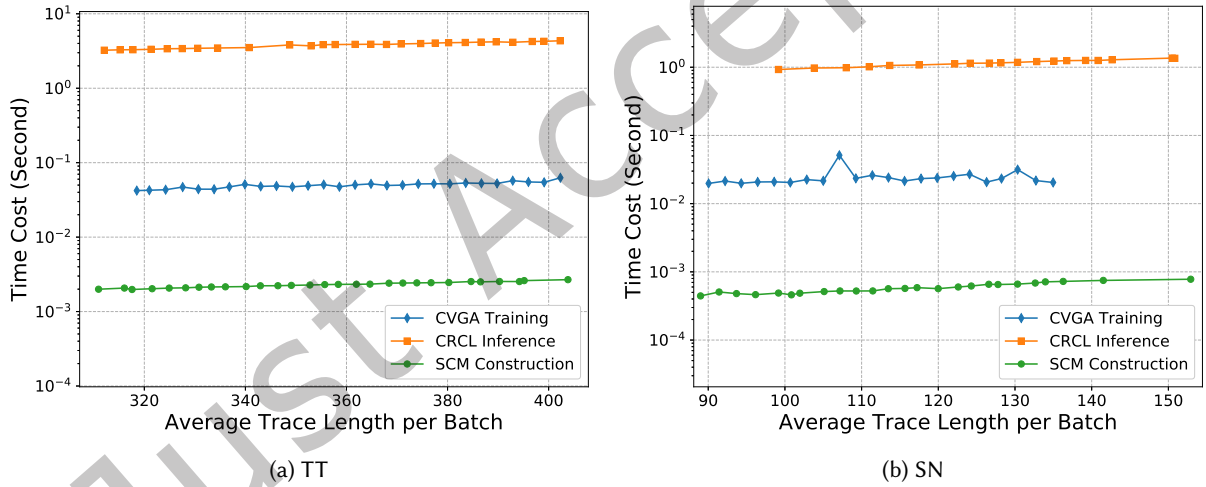


Fig. 5. Time cost of key stages in CRFD under different trace lengths.

As shown in Fig. 5, the time cost of the three stages slowly increases with average trace length per batch. Specifically, SCM Construction remains around 0.001 seconds, which indicates that it is insensitive to trace lengths. CVGA Training shows a slight increase, while CRCL Inference accounts for the majority of the time cost but less than 3 seconds. Similarly, Fig. 6 shows that the time cost of the three stages exhibits a stable or near-linear increase when the average service number per batch increases. Specifically, the time cost of SCM Construction is less than 0.01 seconds, which is negligible. The time cost of CVGA Training and CRCL Inference rises slightly with average service number per batch, but is less than 0.1 and 3 seconds, respectively. In summary, the evaluation

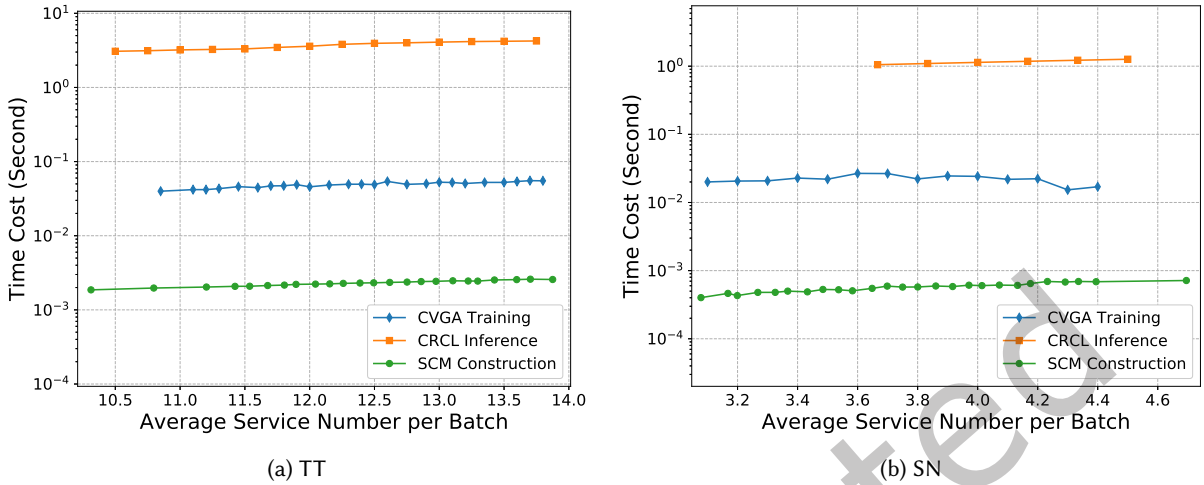


Fig. 6. Time cost of key stages in CRFD under different system sizes.

results validate the scalability of CRFD. Although CRCL Inference contributes the most to time cost, it increases slowly and stays below the telemetry collection interval. Therefore, it does not become a bottleneck for timely fault diagnosis. Furthermore, CRFD significantly improves the effectiveness of fault diagnosis compared with lightweight baselines while maintaining an acceptable time cost.

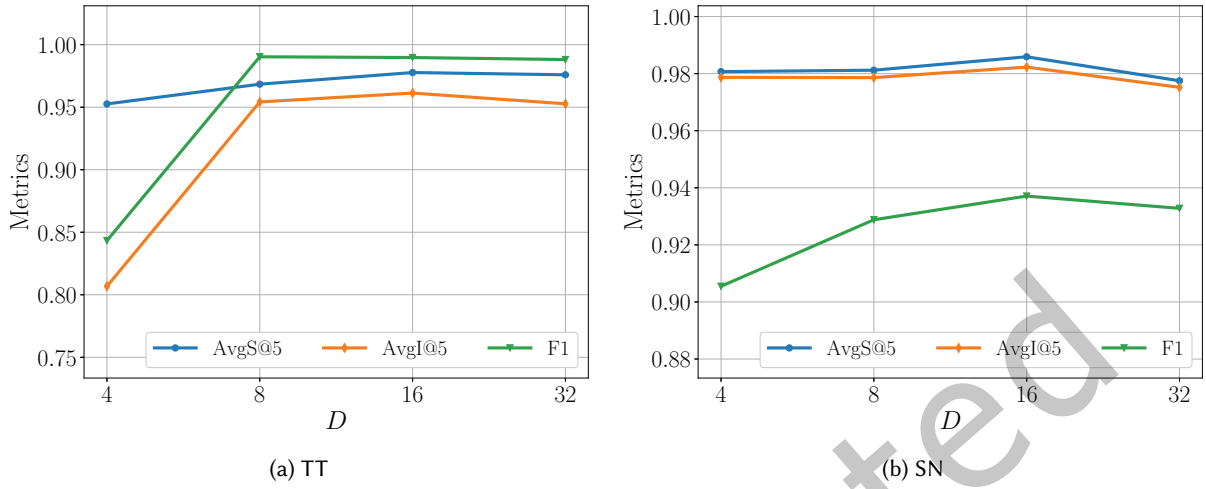
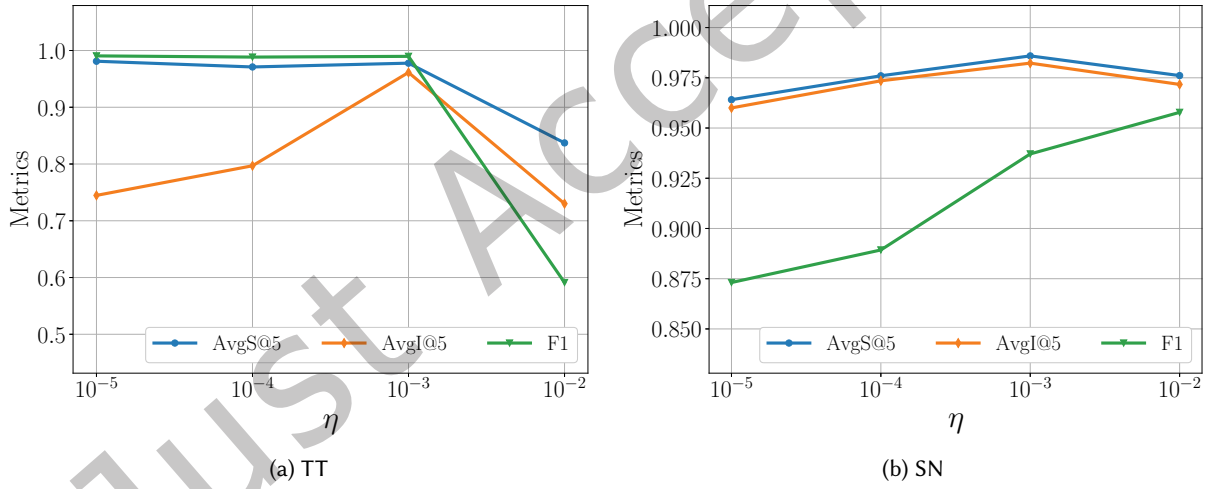
4.10 Hyperparameter Sensitivity (RQ9)

In CRFD, there are three important hyperparameters: latent dimension D , learning rate η and desired false positive rate ε . All hyperparameter tuning and sensitivity analysis are conducted on the validation set.

To evaluate the sensitivity of D , experiments are conducted on two datasets when η and ε are fixed. Fig. 7 shows the impact of D . As shown in Fig. 7, the AvgS@5, AvgI@5 and F1 of CRFD first increase and then start to drop or stabilize with D , and achieve the best overall performance at $D = 16$ on both datasets. This is because CRFD needs a suitable latent dimension to capture more useful information. A larger D may introduce redundant information. Therefore, the choice of D must balance information preservation and redundancy control.

We explore the sensitivity of η on two datasets when D and ε are fixed. Fig. 8 shows the impact of η . On the TT dataset, the AvgS@5 and F1 of CRFD remain stable when $\eta < 10^{-3}$, but sharply drop when $\eta > 10^{-3}$. The AvgI@5 of CRFD first rises and then starts to drop with η , and achieves its peak at $\eta = 10^{-3}$. On the SN dataset, the F1 of CRFD increases with η , while its AvgS@5 and AvgI@5 first increase and then start to decrease, and reach their peaks at $\eta = 10^{-3}$. Comprehensively considering two datasets and three evaluation metrics, CRFD achieves the best overall performance at $\eta = 10^{-3}$. This is because CRFD requires a suitable learning rate to make it converge better.

We check the sensitivity of ε on two datasets when D and η are fixed. Fig. 9 shows the impact of ε . On both datasets, the F1 of CRFD generally decreases as ε increases. For AvgS@5 and AvgI@5, they first decrease and then increase with ε on the TT dataset, while they remain relatively stable on the SN dataset. Comprehensively considering two datasets and three evaluation metrics, CRFD achieves the best overall performance at $\varepsilon = 0.5$. The reason is that the choice of ε determines the strictness of anomaly detection. A lower ε results in a stricter

Fig. 7. Impact of hyperparameter D .Fig. 8. Impact of hyperparameter η .

threshold, while a higher ε leads to a looser threshold. Therefore, the choice of ε is crucial to balance false negatives and false positives.

4.11 Threats to Validity

This section introduces the threats to internal validity and external validity of CRFD, and the measures of mitigating these threats.

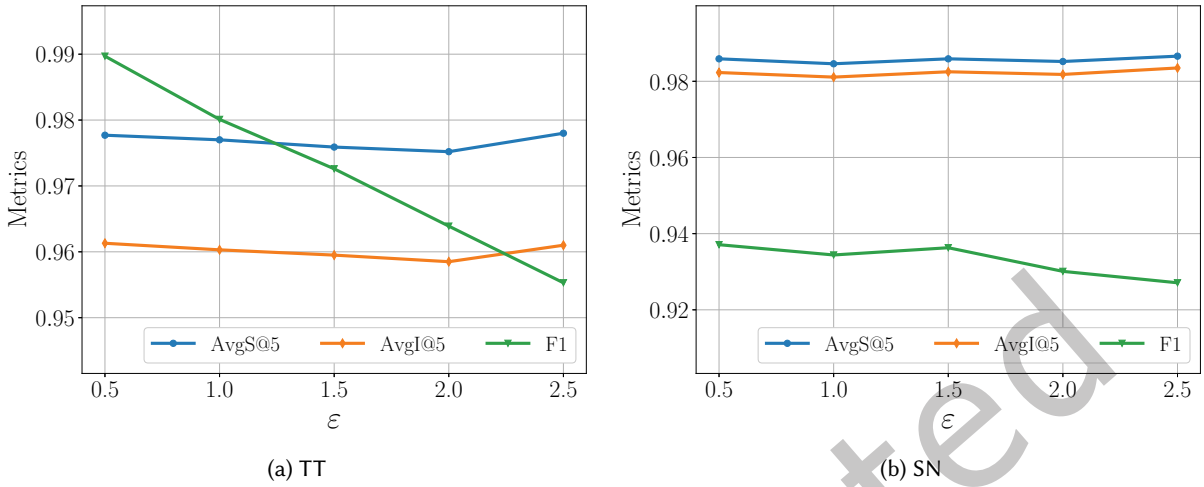


Fig. 9. Impact of hyperparameter ϵ .

The threats to internal validity mainly lie in the data collection, the design of causal computation and the implementations of baselines. (1) CRFD requires designing counterfactual interventions based on NDPL. NDPL is built on historical normal data, and its coverage of normal behaviors determines the reliability of intervention operations. When some faults occur and the corresponding normal patterns are missing from NDPL, the generated counterfactual interventions may only approximate the expected normal behaviors through similar patterns in NDPL. To mitigate this threat, we collect normal data that can cover diverse and representative user requests to ensure sufficient coverage of normal behavior patterns. (2) In microservice systems, multi-layer dependencies are common, and CRFD only performs single-layer causal computation for each variable. Therefore, it may fail to fully capture multi-hop fault propagation paths, which leads to a reduction in the accuracy of fault diagnosis. To mitigate this threat, we leverage CVGA to integrate upstream and downstream information to capture indirect effects. Meanwhile, we use counterfactual reasoning to evaluate the effect of intervention operation on global anomalies, which can effectively compensate for the shortcomings of multi-hop fault propagation. (3) Most baselines require development based on our understanding of their articles since they do not provide open-source code, except DeepLog, LogAnomaly, TraceAnomaly, CausalRCA and Nezha. But our understanding may not be accurate. To mitigate this threat, we carefully follow their articles to develop them and use the best hyperparameter configurations to conduct comparison experiments.

The threat to external validity mainly comes from the experimental environment. In our study, two datasets are used to evaluate all approaches. Although two datasets are collected from two widely used benchmark systems, the effectiveness of CRFD still needs to be verified in real industrial microservice systems. However, it is reasonable to expect that CRFD would achieve a good performance since the number of events and representative fault cases are comparable to those in real production systems.

5 Related works

5.1 Anomaly Detection

Recently, there has been a large number of studies about anomaly detection for microservice systems. They are divided into four classes: metric-based anomaly detection, log-based anomaly detection, trace-based anomaly detection and multi-source-data-based anomaly detection.

Metric-based anomaly detection. Metric-based anomaly detection approaches usually learn the normal patterns of metrics and detect anomalies by identifying outlier metrics. Donut [35] uses a Variational Autoencoder to learn the normal patterns of metrics, and detect anomalies by identifying outlier metrics. MicroRCA [33] uses BIRCH [8] to cluster the response times of service instances, and treat slow response times as anomalies. However, these approaches only consider metrics, thereby missing anomalies reflected in traces and logs.

Log-based anomaly detection. Log-based anomaly detection approaches commonly learn the normal patterns of log events and detect anomalies by predicting the next log event. DeepLog [4] first uses Spell [3] to parse logs into log events. Then, it uses LSTM to learn the sequential patterns of log events, and detects anomalies by predicting the next log event. Similarly, LogAnomaly [20] first uses FT-Tree [42] to parse logs into log events. Then, it combines Template2Vec with LSTM to learn the sequential and quantitative patterns of log events, and detects anomalies by predicting the next log event. However, these approaches only consider logs, thereby missing anomalies reflected in metrics and traces.

Trace-based anomaly detection. Trace-based anomaly detection approaches usually learn the normal patterns of traces and detect anomalies by identifying outlier call paths. TraceAnomaly [17] first encodes a trace as an STV, where each dimension represents a call path and its value describes the corresponding response time. Then, it uses a VAE with posterior flow to learn the normal patterns of STVs, and detect anomalies by identifying outlier STVs. TraceSieve [43] first uses the autoencoder architecture within the adversarial training framework to filter abnormal data from raw data. Then, it combines VGAE with the elastic weight consolidation method to learn normal call patterns, and detect anomalies by identifying outlier call paths. However, these approaches only consider traces, thereby missing anomalies reflected in metrics and logs.

Multi-source-data-based anomaly detection. Multi-source-data-based anomaly detection approaches usually learn the behavior patterns of multi-source observability data and detect anomalies by identifying outlier behaviors. DeepTraLog [39] first constructs a unified graph representation based on traces and logs. Then, it combines GGNN [14] with Deep SVDD [26] to train a one-class classification model for detecting anomalies. iTCRL-LOF [31] first constructs a graph by combining traces and logs. Then, it combines iTCRL with the LOF algorithm to train an anomaly detection model. However, these approaches cannot make full use of the causal relationships in multi-source observability data.

5.2 Root Cause Localization

At present, many studies about root cause localization for microservice systems have been made. They are divided into four classes: metric-based root cause localization, log-based root cause localization, trace-based root cause localization and multi-source-data-based root cause localization.

Metric-based root cause localization. Metric-based root cause localization approaches usually localize root causes by mining the correlation relationships between metrics. TBAC [19] first combines the response time with the dependency relation graph to calculate anomaly correlation. Then, it localizes root cause services based on anomaly correlation. MonitorRank [12] first combines the historical metrics with the latest metrics to construct call graphs. Then, it uses a random walk on call graphs to localize root cause services. CloudRanger [32] uses the PC algorithm [30] to construct impact graphs based on metrics, and uses the second-order random walk on impact graphs to localize root cause services. CausalRCA [34] uses the gradient-based causal structure learning

method [38, 44] to learn causal graphs, and PageRank to localize root cause metrics. However, these approaches only consider metrics, which makes them helpless against code defects and invocation anomalies.

Log-based root cause localization. Log-based root cause localization approaches commonly localize root causes by mining the normal patterns of log events. SBLD [25] uses the spectrum algorithm [11] to extract events from logs and localize root cause operations by comparing the coverage of events. LogCluster [16] uses log clustering to mine the frequent patterns of log sequences and localizes root cause operations by comparing the abnormal log sequences with normal frequent patterns. However, these approaches only consider logs, which makes them helpless against resource issues and invocation anomalies.

Trace-based root cause localization. Trace-based root cause localization approaches usually localize root causes by mining normal call paths. TraceAnomaly [17] mines the normal call paths from traces, and localizes root cause services by comparing the normal call paths with the abnormal call paths. MicroRank [36] first constructs call graphs and operation-trace graphs. Then, it generates normal scores and abnormal scores for each service based on normal traces and abnormal traces. Finally, it combines PageRank with the spectrum algorithms to localize root cause services. However, these approaches only consider traces, which makes them helpless against code defects and resource issues.

Multi-source-data-based root cause localization. Multi-source-data-based root cause localization approaches commonly localize root causes by mining normal behavior patterns of multi-source observability data. DiagFusion [41] first combines lightweight preprocessing and representation learning to map multi-source observability data into the same latent space. Then, it uses a GNN to learn historical fault patterns and localizes root cause services based on this pattern. Nezha [37] first extracts event patterns from multi-source observability data. Then, it localizes root cause resources or code regions by comparing event patterns in the fault-free phase with those in the fault-suffering phase. TrinityRCL [7] first constructs causal graphs based on multi-source observability data. Then, it combines the dynamic time warping [1] with the first-order temporal correlation [2] to calculate the anomaly scores between nodes. Finally, it uses the random walk with restart algorithm [22] to localize root causes. However, these methods lack sufficient causality and interpretability, which makes it impossible to investigate the root causes behind superficial phenomena and provide effective guidance for fault mitigation.

6 Conclusion

In this study, we present CRFD, a causal and interpretable fault diagnosis approach based on multi-source observability data for microservice systems. This approach uses counterfactual reasoning to diagnose faults and provide effective guidance for subsequent troubleshooting. CRFD first transforms multi-source observability data into SCMs. Then, it uses a Causal Variational Graph Autoencoder to learn SCMs, and detects anomalies based on SCMs. When an anomaly is detected, counterfactual reasoning is applied on SCMs to localize root causes and tell operators why they are culprits and how to deal with them to effectively mitigate faults. Experimental results on two datasets show that CRFD outperforms all baselines in terms of anomaly detection and root cause localization.

In the future, we will explore how to hierarchically or approximately extend multi-layer causal computation while maintaining the causal consistency of SCMs. Furthermore, we will design an anomaly detector that can provide more accurate and fine-grained normal probabilities of microservice systems to help the more accurate fault diagnosis.

Acknowledgments

This work was supported by the National Natural Science Foundation Project of China under Grants 62472329 and 62072342, and by the National Key Research and Development Plan Project of China under Grant 2022YFB3304300.

References

- [1] Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*. 359–370.
- [2] Ahlame Douzal Chouakria and Panduranga Naidu Nagabhushan. 2007. Adaptive dissimilarity index for measuring time series proximity. *Advances in Data Analysis and Classification* 1 (2007), 5–21.
- [3] Min Du and Feifei Li. 2016. Spell: Streaming parsing of system event logs. In *Proceedings of the 16th International Conference on Data Mining*. 859–864.
- [4] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1285–1298.
- [5] elastic. 2024. Elasticsearch. <https://www.elastic.co/elasticsearch/>. Retrieved November 7, 2024.
- [6] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rath, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. 3–18.
- [7] Shenghui Gu, Guoping Rong, Tian Ren, He Zhang, Haifeng Shen, Yongda Yu, Xian Li, Jian Ouyang, and Chunan Chen. 2023. TrinityRCL: Multi-Granular and Code-Level Root Cause Localization Using Multiple Types of Telemetry Data in Microservice Systems. *IEEE Transactions on Software Engineering* 49, 5 (2023), 3071–3088.
- [8] Anton Gulenko, Florian Schmidt, Alexander Acker, Marcel Wallschläger, Odej Kao, and Feng Liu. 2018. Detecting Anomalous Behavior of Black-Box Services Modeled with Distance-Based Online Clustering. In *Proceedings of the 11th International Conference on Cloud Computing*. 912–915.
- [9] Xiao Han, Lu Zhang, Yongkai Wu, and Shuhan Yuan. 2023. On Root Cause Localization and Anomaly Mitigation through Causal Inference. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 699–708.
- [10] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. 2017. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In *Proceedings of the 2017 IEEE International Conference on Web Services*. 33–40.
- [11] James A Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of test information to assist fault localization. In *Proceedings of the 24th international conference on Software engineering*. 467–477.
- [12] Myunghwan Kim, Roshan Sumbaly, and Sam Shah. 2013. Root cause detection in a service-oriented architecture. *ACM SIGMETRICS Performance Evaluation Review* 41, 1 (2013), 93–104.
- [13] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *Proceedings of the 45th International Conference on Software Engineering*. 1750–1762.
- [14] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2016. Gated graph sequence neural networks. In *Proceedings of the 4th International Conference on Learning Representations*.
- [15] JinJin Lin, Pengfei Chen, and Zibin Zheng. 2018. Microscope: Pinpoint performance issues with causal graphs in micro-service environments. In *Proceedings of the 16th International Conference on Service-Oriented Computing*. 3–20.
- [16] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuwei Chen. 2016. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 102–111.
- [17] Ping Liu, Haowen Xu, Qianyu Ouyang, Rui Jiao, Zhekang Chen, Shenglin Zhang, Jiahai Yang, Linlin Mo, Jice Zeng, Wenman Xue, and Dan Pei. 2020. Unsupervised Detection of Microservice Trace Anomalies through Service-Level Deep Bayesian Networks. In *Proceedings of the 31st International Symposium on Software Reliability Engineering*. 48–58.
- [18] Meng Ma, Weilan Lin, Disheng Pan, and Ping Wang. 2022. ServiceRank: Root Cause Identification of Anomaly in Large-Scale Microservice Architectures. *IEEE Transactions on Dependable and Secure Computing* 19, 5 (2022), 3087–3100.
- [19] Nina Marwede, Matthias Rohr, André van Hoorn, and Wilhelm Hasselbring. 2009. Automatic Failure Diagnosis Support in Distributed Large-Scale Software Systems Based on Timing Behavior Anomaly Correlation. In *Proceedings of the 13th European Conference on Software Maintenance and Reengineering*. 47–58.
- [20] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, Vol. 19. 4739–4745.
- [21] OpenTelemetry.io. 2025. OpenTelemetry. <https://opentelemetry.io/>.
- [22] Jia-Yu Pan, Hyung-Jeong Yang, Christos Faloutsos, and Pinar Duygulu. 2004. Automatic multimedia cross-modal correlation discovery. In *Proceedings of the 10th ACM SIGKDD international conference on Knowledge discovery and data mining*. 653–658.
- [23] Judea Pearl. 2009. *Causality*. Cambridge university press.
- [24] Prometheus. 2024. Prometheus. <https://github.com/prometheus/>. Retrieved November 7, 2024.

- [25] Carl Martin Rosenberg and Leon Moonen. 2020. Spectrum-based log diagnosis. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–12.
- [26] Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. 2018. Deep one-class classification. In *Proceedings of the 35th International conference on machine learning*. 4393–4402.
- [27] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing and Management* 24, 5 (1988), 513–523.
- [28] Pablo Sánchez-Martin, Miriam Rateike, and Isabel Valera. 2022. Vaca: Designing variational graph autoencoders for causal queries. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, Vol. 36. 8159–8168.
- [29] skywalking.apache.org. 2024. Apache SkyWalking. <http://skywalking.apache.org/>. Retrieved November 7, 2024.
- [30] Peter Spirtes, Clark Glymour, and Richard Scheines. 2001. *Causation, prediction, and search*. MIT press.
- [31] Xiangbo Tian, Shi Ying, Tiangang Li, Mengting Yuan, Ruijin Wang, Yishi Zhao, and Jianga Shang. 2024. iTCRL: Causal-Intervention-Based Trace Contrastive Representation Learning for Microservice Systems. *IEEE Transactions on Software Engineering* 50, 10 (2024), 2583–2601.
- [32] Ping Wang, Jingmin Xu, Meng Ma, Weilan Lin, Disheng Pan, Yuan Wang, and Pengfei Chen. 2018. CloudRanger: Root Cause Identification for Cloud Native Systems. In *Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 492–502.
- [33] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *Proceedings of the 2020 IEEE/IFIP Network Operations and Management Symposium*. 1–9.
- [34] Ruyue Xin, Peng Chen, and Zhiming Zhao. 2023. CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software* 203 (2023), 111724.
- [35] Haowen Xu, Wenxiao Chen, Nengwen Zhao, Zeyan Li, Jiahao Bu, Zhihan Li, Ying Liu, Youjian Zhao, Dan Pei, Yang Feng, Jie Chen, Zhaogang Wang, and Honglin Qiao. 2018. Unsupervised Anomaly Detection via Variational Auto-Encoder for Seasonal KPIs in Web Applications. In *Proceedings of the 2018 World Wide Web Conference*. 187–196.
- [36] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *Proceedings of the 2021 Web Conference*. 3087–3098.
- [37] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 553–565.
- [38] Yue Yu, Jie Chen, Tian Gao, and Mo Yu. 2019. DAG-GNN: DAG structure learning with graph neural networks. In *Proceedings of the 36th International conference on machine learning*. 7154–7163.
- [39] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeepTraLog: trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th International Conference on Software Engineering*. 623–634.
- [40] Chenxi Zhang, Xin Peng, Tong Zhou, Chaofeng Sha, Zhenghui Yan, Yiru Chen, and Hong Yang. 2022. TraceCRL: contrastive representation learning for microservice trace analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1221–1232.
- [41] Shenglin Zhang, Pengxiang Jin, Zihan Lin, Yongqian Sun, Bicheng Zhang, Sibao Xia, Zhengdan Li, Zhenyu Zhong, Minghua Ma, Wa Jin, Dai Zhang, Zhenyu Zhu, and Dan Pei. 2023. Robust Failure Diagnosis of Microservice System Through Multimodal Data. *IEEE Transactions on Services Computing* 16, 6 (2023), 3851–3864.
- [42] Shenglin Zhang, Weibin Meng, Jiahao Bu, Sen Yang, Ying Liu, Dan Pei, Jun Xu, Yu Chen, Hui Dong, Xianping Qu, et al. 2017. Syslog processing for switch failure diagnosis and prediction in datacenter networks. In *Proceedings of the 25th International Symposium on Quality of Service*. 1–10.
- [43] Shenglin Zhang, Zhongjie Pan, Heng Liu, Pengxiang Jin, Yongqian Sun, Qianyu Ouyang, Jiaju Wang, Xueying Jia, Yuzhi Zhang, Hui Yang, Yongqiang Zou, and Dan Pei. 2023. Efficient and Robust Trace Anomaly Detection for Large-Scale Microservice Systems. In *Proceedings of the 34th International Symposium on Software Reliability Engineering*. 69–79.
- [44] Xun Zheng, Bryon Aragam, Pradeep Ravikumar, and Eric P Xing. 2018. DAGs with NO TEARS: continuous optimization for structure learning. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. 9492–9503.
- [45] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* 47, 2 (2021), 243–260.

Received 8 March 2025; revised 30 January 2026; accepted 3 February 2026